Drawer memory: live experiment report

LUT-addressed memory written at inference time as a replacement for attention. Generated 2026-10-20 12:07 from 105 finished runs; abstract last revised 2026-10-20 12:10.

Abstract

Drawer memory — a hash-addressed associative memory written at inference, using spiky's LUT addressing (anchor-pair sign bits → table cells) — replaces softmax attention: each token stores its value in a few table cells chosen by fixed yes/no comparisons, and later tokens read the cells their own comparisons select. On associative recall it is close to linear attention up to 64 pairs (within 0–1.6 points) and beats it at equal parameters; on real text (enwik8) hard drawers are 2.5% behind linear attention at 5.4× lower cost per token (measured, CPU), and a **bounded soft read** (each token reads and writes its 4 most likely cells per table instead of one) halves that gap (to 1.4–1.6%) at 2.2–2.7× lower cost; against softmax with a KV cache the saving is ~115× at 16k context. Two design rules make it work: one shared address projection, and many small (4-bit) tables; soft-to-hard training largely fixes slow learning. Open: behaviour at real scale and with a fused kernel, which need a GPU.

Key findings — current state of knowledge

Drawer memory stores each token's information in a few cells of fixed-size tables, chosen by yes/no comparisons of the token's own numbers (spiky's LUT addressing); later tokens read the cells their own comparisons point to. Mathematically it is linear attention with a sparse, binary feature map; per token it touches a fixed number of rows.

Recall (synthetic MQAR)

- **It works.** 99.8–100% recall at 16–32 key–value pairs, comparable to linear attention of the same memory size (99.97–99.99%).
- **64 pairs.** Drawers 16×4 (memory 256) start learning at step ~1,250, earlier than linear attention (~2,250), and reach 98.3% / 99.1% vs 99.9%. With memory 64, 10,000 steps give 95.4% (hard) and 97.1% (neighbour cell) vs 99.9%.
- **Equal parameters.** Linear attention with ~the same parameter count (F=32) reaches 96.9% / 97.4% at 32 pairs vs 99.8% for drawers, at the same measured speed. On text this advantage does not hold (Shakespeare: 2.225 vs 2.238 bpc).
- **Non-adjacent pairs (2 filler tokens).** Plain drawers learn late (step ~2,250) and reach 90–91% vs 99.99% for linear attention; softmax fails in this setting with or without RoPE

(3.4–4.3%). Soft-to-hard addressing lifts drawers to 96.3–99.8% at no inference cost.
- **Memory editing (8 of 16 keys re-assigned).** Plain additive writes: 98.0–98.2% on updated keys, the same as linear attention (98.0%; softmax 99.99%). Decay writes are worse (93.8–96.5%); overwrite-style writes do not learn (4.2%) because every token overwrites shared cells.

Design rules

- **One shared address projection.** With separate query/key projections learning never starts (0.4%).
- **Many small tables.** 4 bits (16 cells) per table is the sweet spot: 6-bit tables learn late or not at all and depend on the anchor draw; 2-bit tables hit a collision ceiling (77–79% at 32 pairs, 27.7% at 64 pairs with memory 64).
- **Failures were learning speed, not capacity.** A run at 0.45% after 3,000 steps reached 100% with 10,000.
- **Soft-to-hard addressing.** Soft cell addresses that harden by 30% of training largely fix slow learning; the final model is fully hard. A higher learning rate is not an alternative: drawers have a narrower learning-rate window than linear attention.
- **Spiky's read tricks.** The neighbour cell helps 6-bit tables, depending on the anchor draw (99.8–99.9% with one, 25.8% with another); the confidence score hurts.
- **What did not help.** Hybrids with a local softmax window (two designs), squared match-count reads, decay or overwrite writes.

Why learning 'clicks' (diagnostics)

- **Address agreement.** In layer 1 the share of tables where a query lands in the cell that stored its value jumps from 19% to 81% between steps 750 and 1,000, exactly when recall jumps; it ends at 93%. Hashing quality is good throughout (entropy 0.96–0.99 of maximum, balanced bits, collisions at chance).
- **Failed 6-bit runs.** Agreement crawls from 2% to 11%; the optimiser correlates the bits (|corr| 0.09 → 0.31), coarsening addresses and quadrupling collisions without reaching agreement.
- **Noisy reads.** The correct value carries only ~11% of a read's weight; later layers still decode it.

Measured cost (CPU, PyTorch, no custom kernel)

- **Per token, equal memory (256).** Drawers 16×4: 5.2–5.7 µs; linear attention: 27–29.5 µs (~5×), flat in context length. Softmax with a KV cache: 20 µs at 256 tokens, 635 µs at 16k (~115× drawers), with 16 MB of state per sequence instead of 132 KB. The calculated 60× arithmetic saving shrinks to ~5× because projections and gather/scatter do not get cheaper with sparsity. Timings vary ±5% between runs.

- **Accuracy vs cost (32 pairs).** Linear: 96.9–97.4% at 5.2 µs (F=32), 99.5% at 7.8, 99.94% at 13.7, 99.98% at 28.2. Drawers: 96.9% at 2.9 µs (F=64), 99.4% at 4.0, 99.8% at 5.4. Up to ~8 µs drawers are more accurate; above 99.9% only linear attention gets there, at 2.5–5× the cost.

Real text

- **enwik8 (clean test, no overfitting).** After 1,500 steps: linear attention 2.341 bpc, hard drawers 2.400 (+2.5%, 5.5 µs vs 29.5), drawers with a **bounded soft read** 2.372–2.379 (+1.4–1.6%, 10.9–13.2 µs). The bounded soft read stores and reads the 4 most likely cells per table (exact top-4, or the cheaper flip-2 choice); a longer soft phase alone does not close the gap.
- **Tiny Shakespeare.** Drawers 2.237–2.239 bpc, between linear attention (2.217) and softmax (2.247–2.251), but all models overfit (~25 passes; training 1.6–1.8 bpc), so this ranking mixes modelling power with regularisation. A local-window-only model reaches 2.47 bpc: drawers' long-range memory is worth ~0.23 bpc.
- **Scale.** A 4× larger model memorises Tiny Shakespeare (training 0.69 vs held-out 3.33 bpc); a scale test needs a large corpus and a GPU.

Target claim (from external review): comparable recall with fewer memory accesses and lower *measured* decoding cost. Status: met at small scale on recall; on real text drawers are 1.4–2.5% behind linear attention at 2.2–5.4× lower measured cost. Open: real scale and a fused kernel (GPU).

key findings · 1 pilot · 2 shared projection · 3 night grid · 4 small tables · speed benchmark · 5 real text & pressure · 6 diagnostics & editing · 7 fair baselines, annealing, cost curve, scale · 8 enwik8 pilot

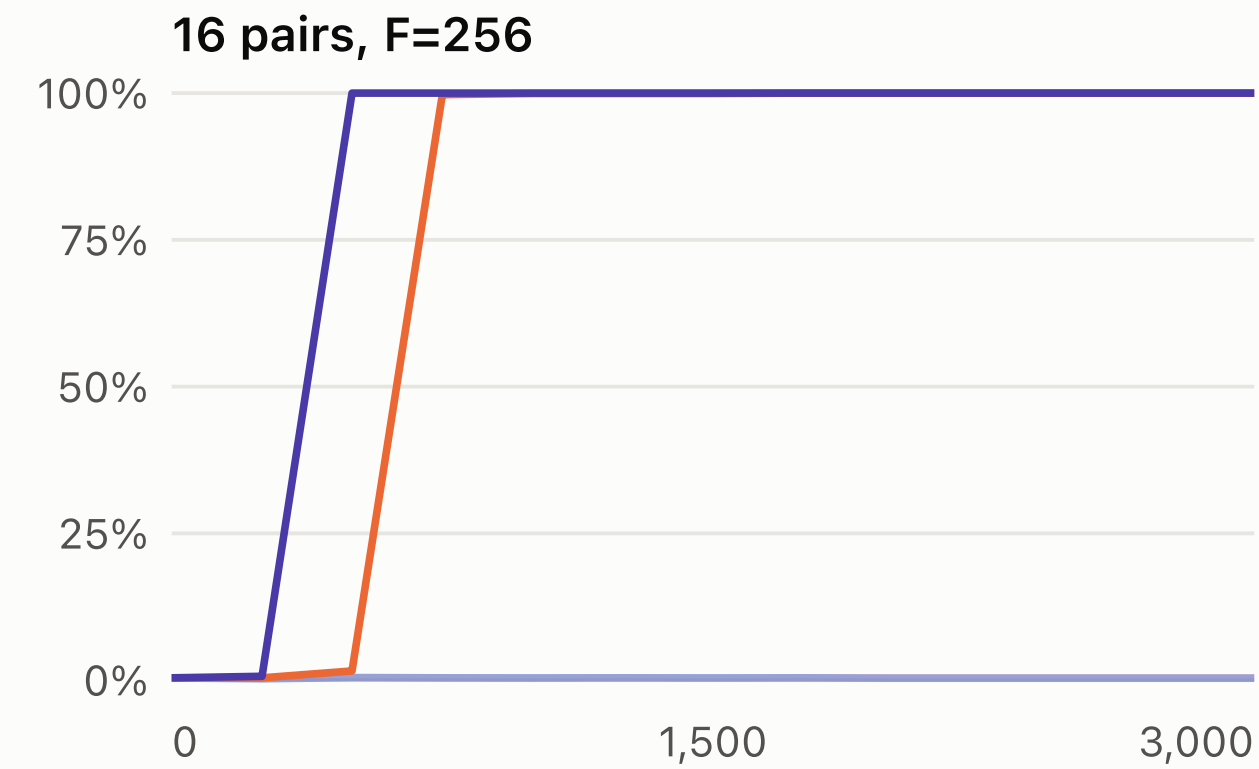
Batch 1 — pilot: does drawer memory learn at all?

4 of 4 runs finished · 2026-10-18 afternoon

Question. At 16 pairs and memory 256, can drawer memory (separate query/key projections) solve recall like softmax and linear attention?

What we learned. No. Both drawer variants stayed at chance (0.3–0.4%) while softmax and linear attention reached ~100%. With separate projections, a query and its key never land in the same cell, so the model gets no signal to learn from.

drawers, hard, untied 4×6b · linear attention · drawers + nbr + conf, untied 4×6b · softmax attention · seed 1 (dashed)



▼ Run table

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
16 pairs, F=256	drawers + nbr + conf, untied 4×6b	0	0.41%	never	559,624	11.0	90.1
16 pairs, F=256	drawers, hard, untied 4×6b	0	0.32%	never	559,618	6.6	88.7
16 pairs, F=256	linear attention	0	99.99%	750	1,018,368	3.8	86.1
16 pairs, F=256	softmax attention	0	100.00%	500	559,616	1.4	73.9

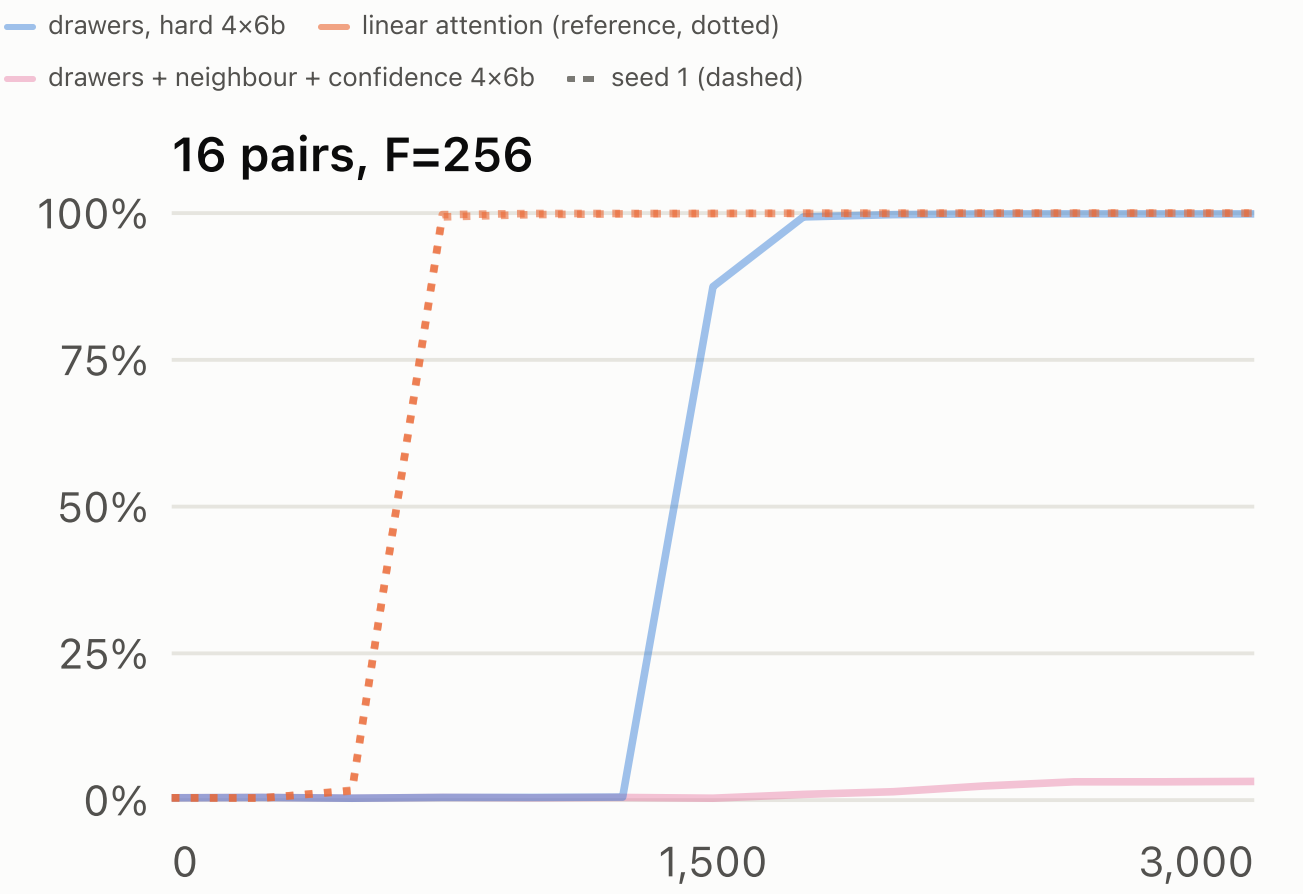
Batch 2 — the fix: one shared address projection

2 of 2 runs finished · 2026-10-18 evening

Question. Does using the same projection for storing and reading (as Reformer does) let drawer memory learn?

What we learned. Yes for the hard variant: 99.9% recall, after an abrupt jump at step ~1,500. The variant with spiky's neighbour cell and confidence score had only started

learning (3.2%) when the 3,000 steps ended.



▼ Run table

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
16 pairs, F=256	drawers + neighbour + confidence 4×6b	0	3.20%	never	526,856	5.8	83.9
16 pairs, F=256	drawers, hard 4×6b	0	99.88%	1500	526,850	4.1	86.1

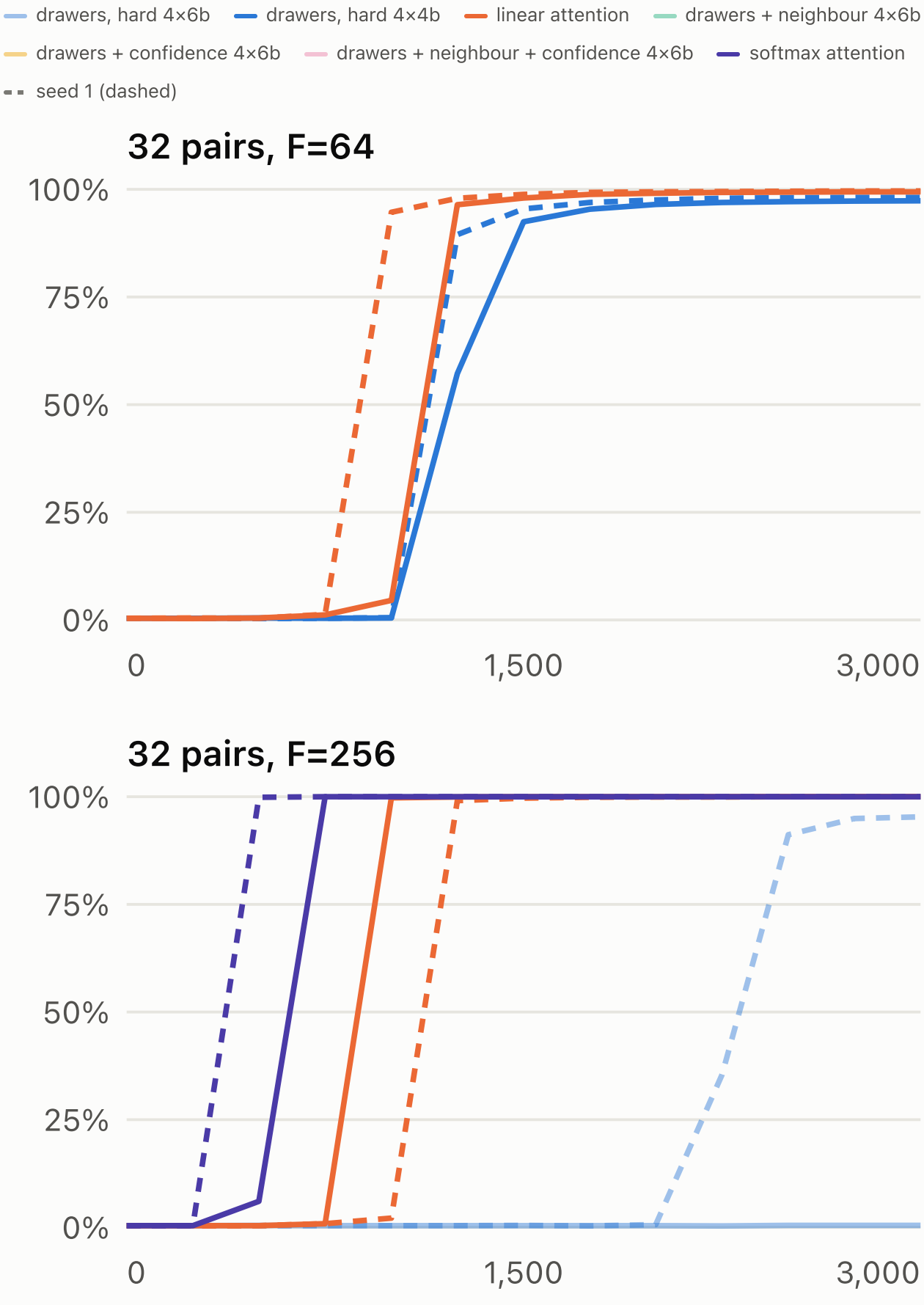
Batch 3 — night grid: drawers vs linear attention under load

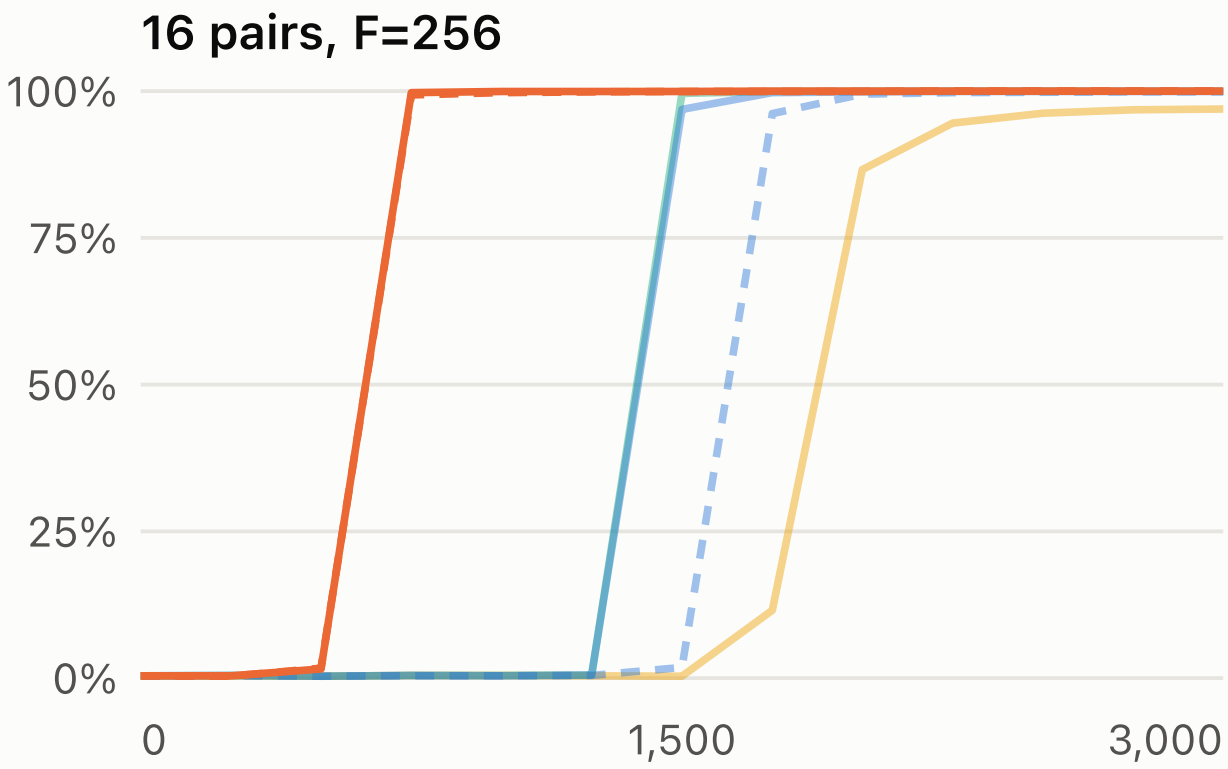
23 of 23 runs finished · night 2026-10-18/19

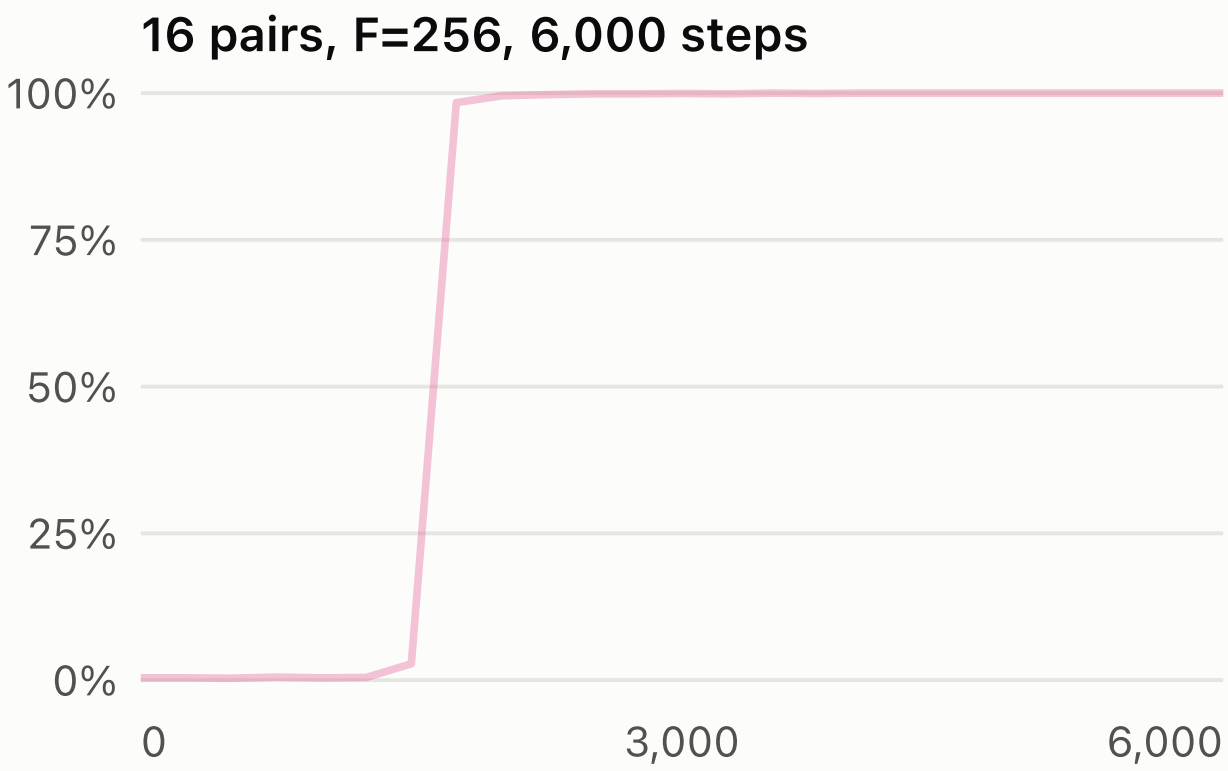
Question. At equal memory (64 or 256 per head) and 16, 32 or 64 pairs, do drawers keep up with linear attention? Which of spiky's tricks help?

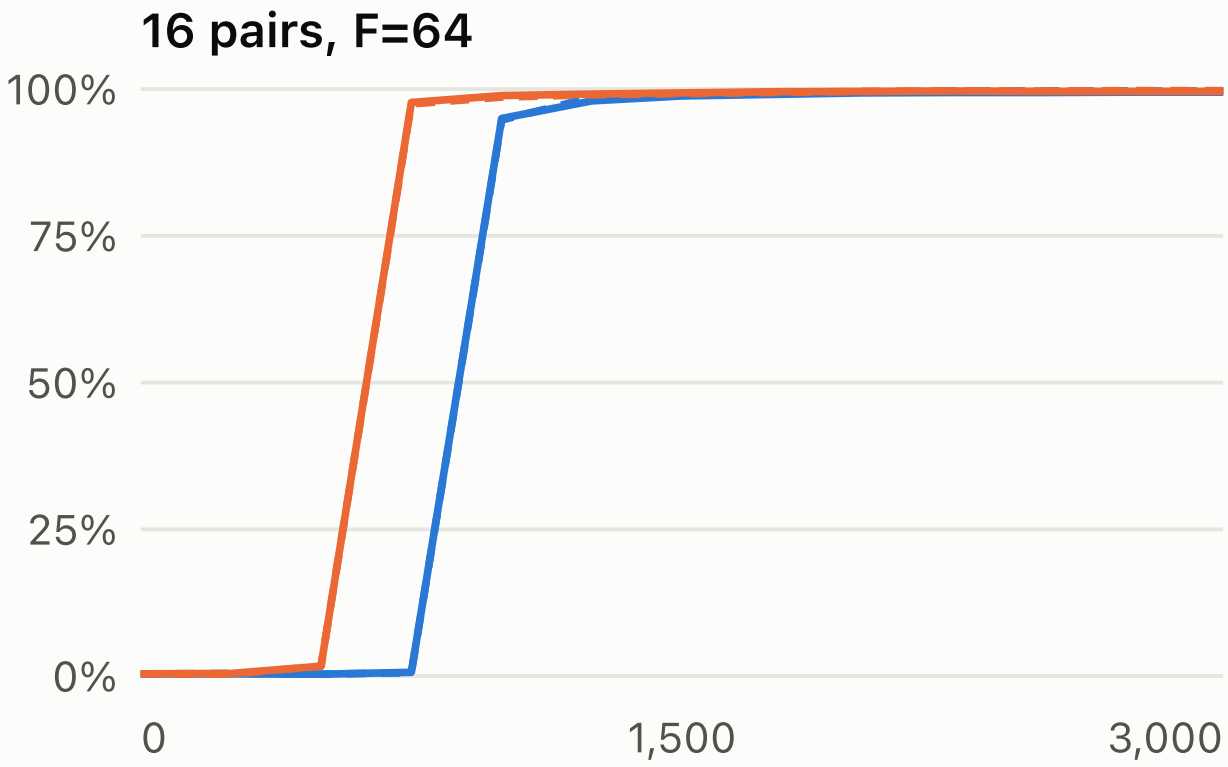
What we learned. Comparable at 16 pairs; behind at 32 pairs with memory 64 (97–98% vs 99.4–99.6%); unstable at 32 pairs with memory 256 (one seed never learned) and

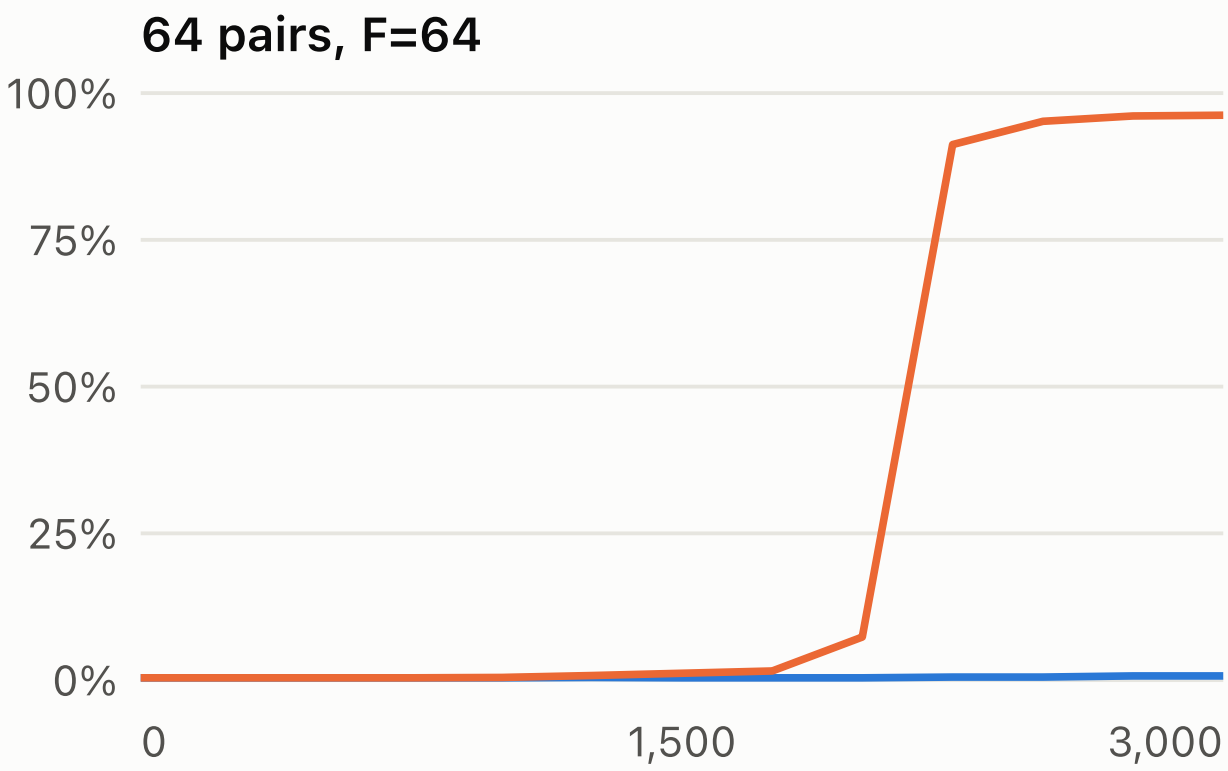
failed at 64 pairs within 3,000 steps. Drawers always learned later than linear attention. The neighbour cell helped; the confidence score slowed learning and lowered accuracy.




16 pairs, F=256
100%
75%
50%
25%
0%
0
1,500
3,000


16 pairs, F=256, 6,000 steps
100%
75%
50%
25%
0%
0
3,000
6,000


16 pairs, F=64
100%
75%
50%
25%
0%
0
1,500
3,000


64 pairs, F=64
100%
75%
50%
25%
0%
0
1,500
3,000

▼ Run table

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
16 pairs, F=256	drawers + confidence 4×6b	0	96.96 %	2000	526,854	4.8	80.4
16 pairs, F=256	drawers + neighbour 4×6b	0	99.99 %	1500	526,852	5.4	79.7
16 pairs, F=256	drawers, hard 4×6b	0	99.92 %	1500	526,850	4.4	79.7
16 pairs, F=256	drawers, hard 4×6b	1	99.86 %	1750	526,850	4.4	80.9
16 pairs, F=256	linear attention	0	99.99 %	750	1,018,368	3.8	86.1
16 pairs, F=256	linear attention	1	99.99 %	750	1,018,368	3.9	85.7
16 pairs, F=256, 6,000 steps	drawers + neighbour + confidence 4×6b	0	99.98 %	1750	526,856	11.8	81.7
16 pairs, F=64	drawers, hard 4×4b	0	99.45 %	1000	526,850	2.8	82.1
16 pairs, F=64	drawers, hard 4×4b	1	99.65 %	1000	526,850	2.8	82.0
16 pairs, F=64	linear attention	0	99.63 %	750	625,152	1.9	85.0
16 pairs, F=64	linear attention	1	99.74 %	750	625,152	2.0	86.2
32 pairs, F=256	drawers, hard 4×6b	0	0.45%	never	526,850	9.0	83.1
32 pairs, F=256	drawers, hard 4×6b	1	95.33 %	2500	526,850	9.0	85.2
32 pairs, F=256	linear attention	0	99.98 %	1000	1,018,368	7.4	86.0
32 pairs, F=256	linear attention	1	99.97 %	1250	1,018,368	7.4	88.5
32 pairs, F=256	softmax attention	0	100.00 %	750	559,616	3.1	87.8

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
32 pairs, F=256	softmax attention	1	100.00 %	500	559,616	3.1	85.3
32 pairs, F=64	drawers, hard 4×4b	0	97.32 %	1250	526,850	6.3	75.5
32 pairs, F=64	drawers, hard 4×4b	1	98.11 %	1250	526,850	6.2	78.2
32 pairs, F=64	linear attention	0	99.42 %	1250	625,152	4.1	75.8
32 pairs, F=64	linear attention	1	99.59 %	1000	625,152	4.1	83.6
64 pairs, F=64	drawers, hard 4×4b	0	0.72%	never	526,850	13.1	87.0
64 pairs, F=64	linear attention	0	96.25 %	2250	625,152	8.6	88.7

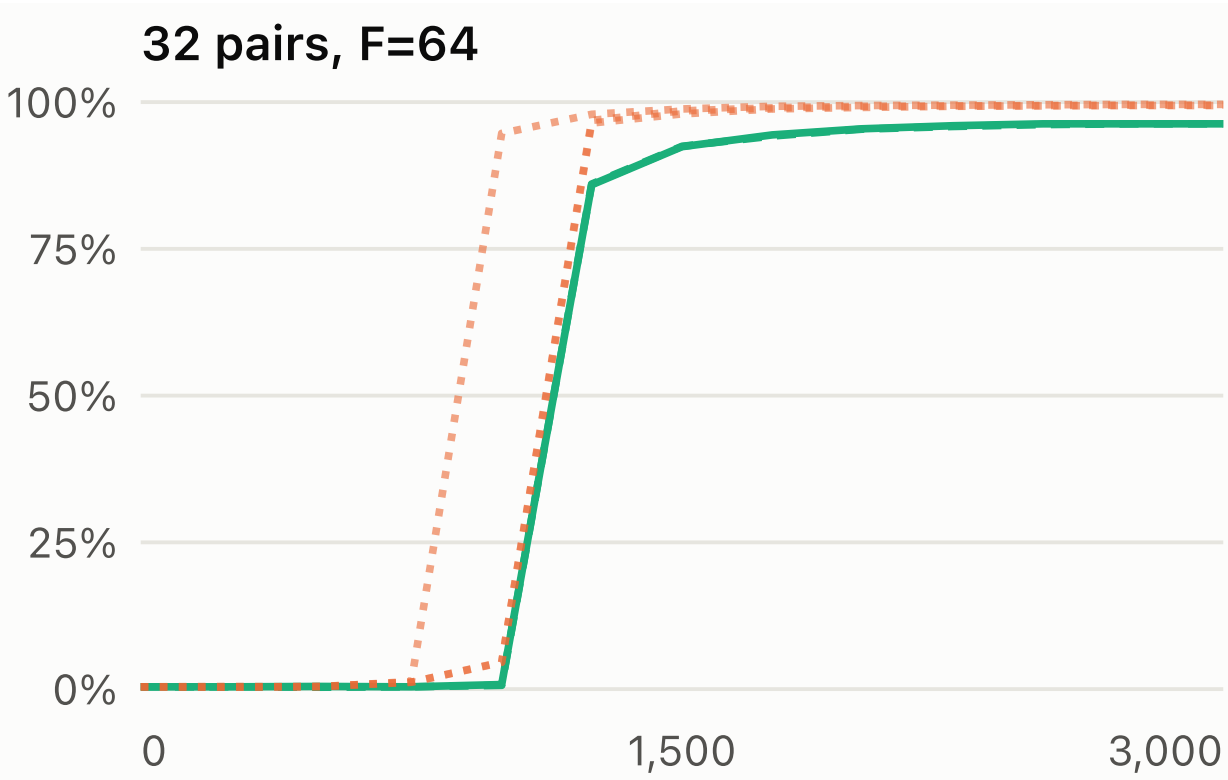
Batch 4 — neighbour cells, small tables, longer training

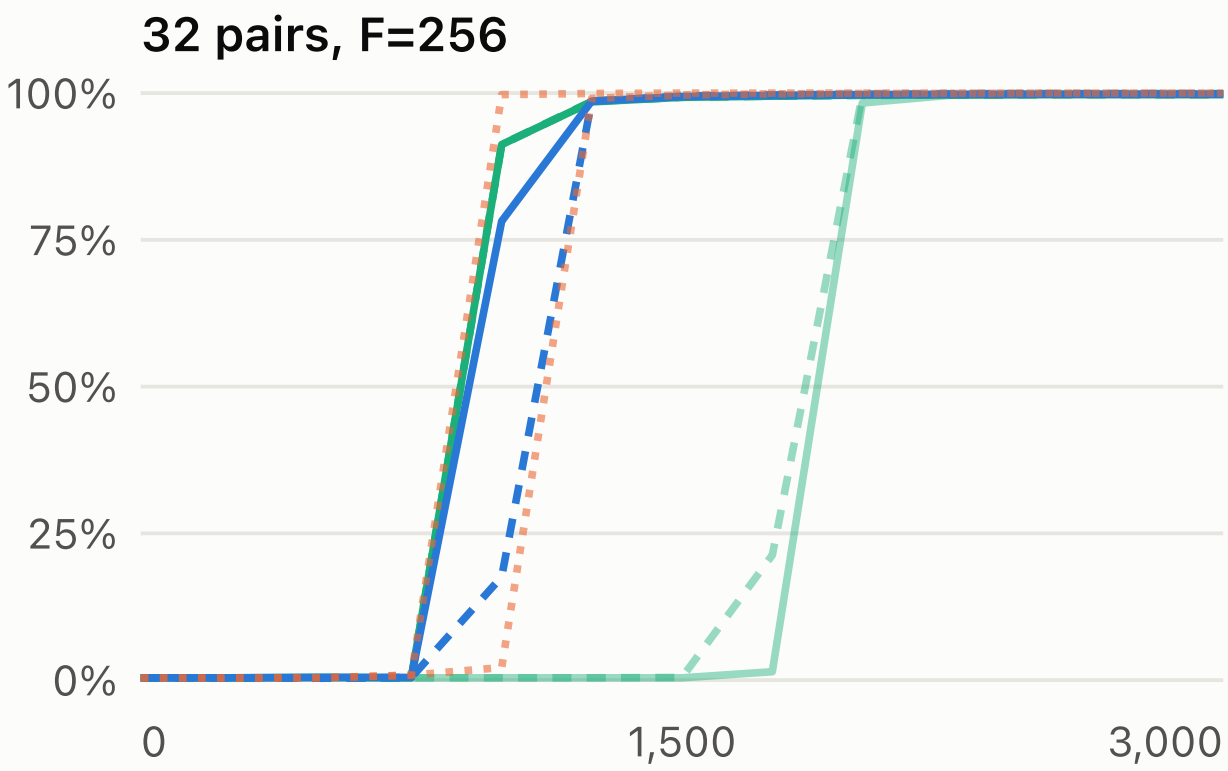
13 of 13 runs finished · 2026-10-19 morning, finished 13:05

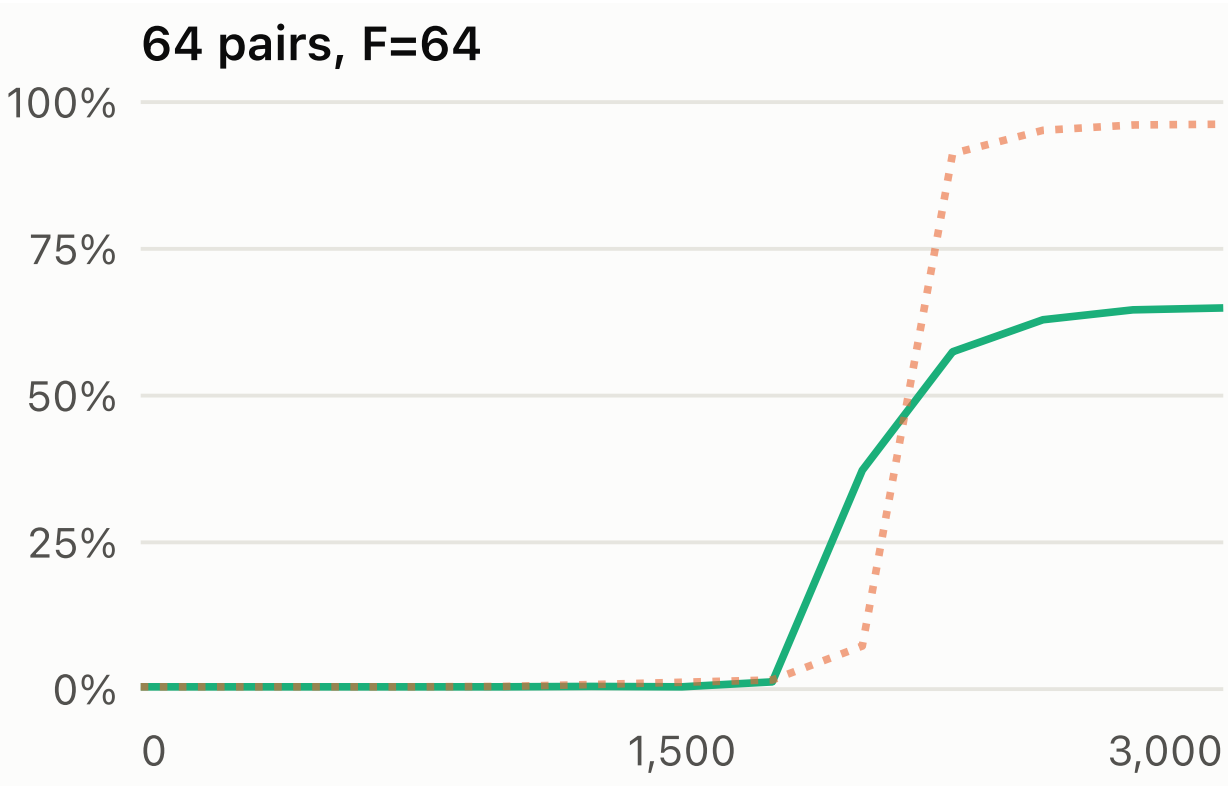
Question. Can the neighbour cell, smaller tables (16 × 4 bits instead of 4 × 6 at the same memory) or 10,000 steps fix the failures?

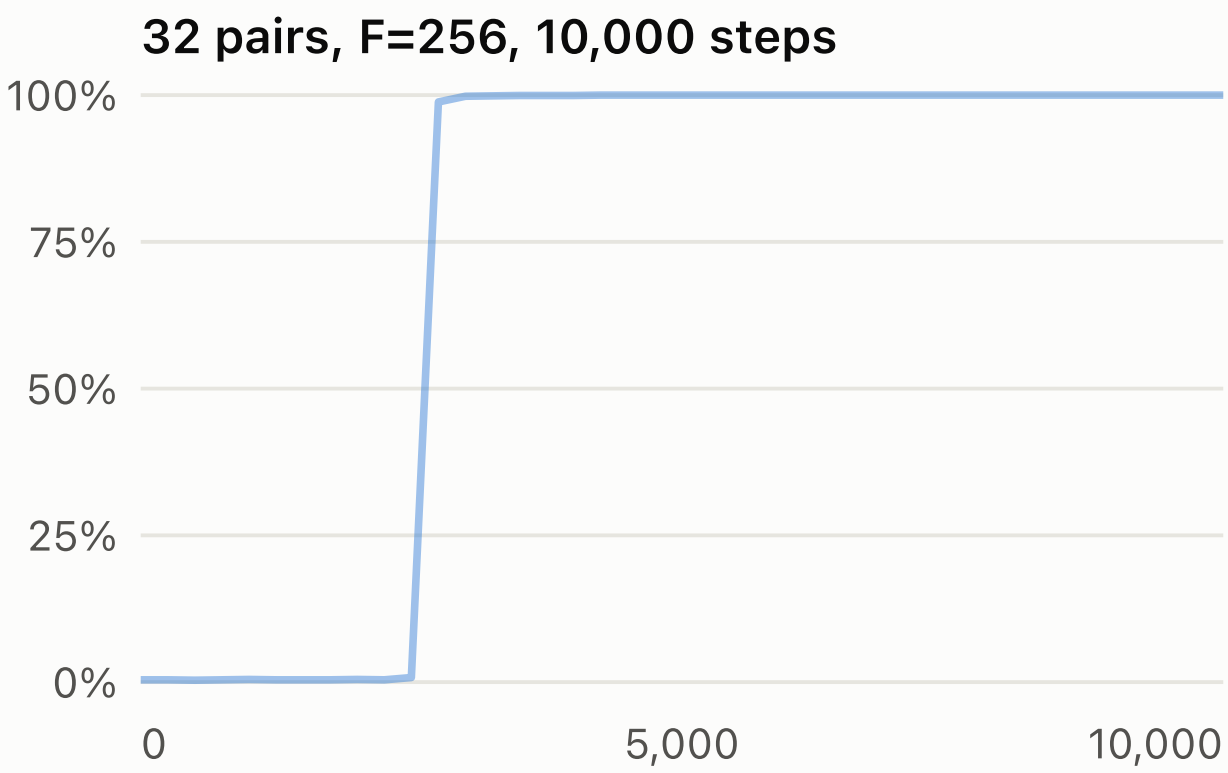
What we learned. All three help. 16 tables × 4 bits learn as early as linear attention and reach 99.8% at 32 pairs; the neighbour cell rescues 6-bit tables with this anchor draw (99.8–99.9%; with another draw only 25.8%, see batch 6); the 32-pair run that never learned in 3,000 steps reaches 100% in 10,000. At 64 pairs with memory 64, 10,000 steps lift hard drawers from chance to 95.4% and neighbour drawers to 97.1% (learning from step ~2,000, as early as linear attention), still below linear attention's 99.9% with the same budget.

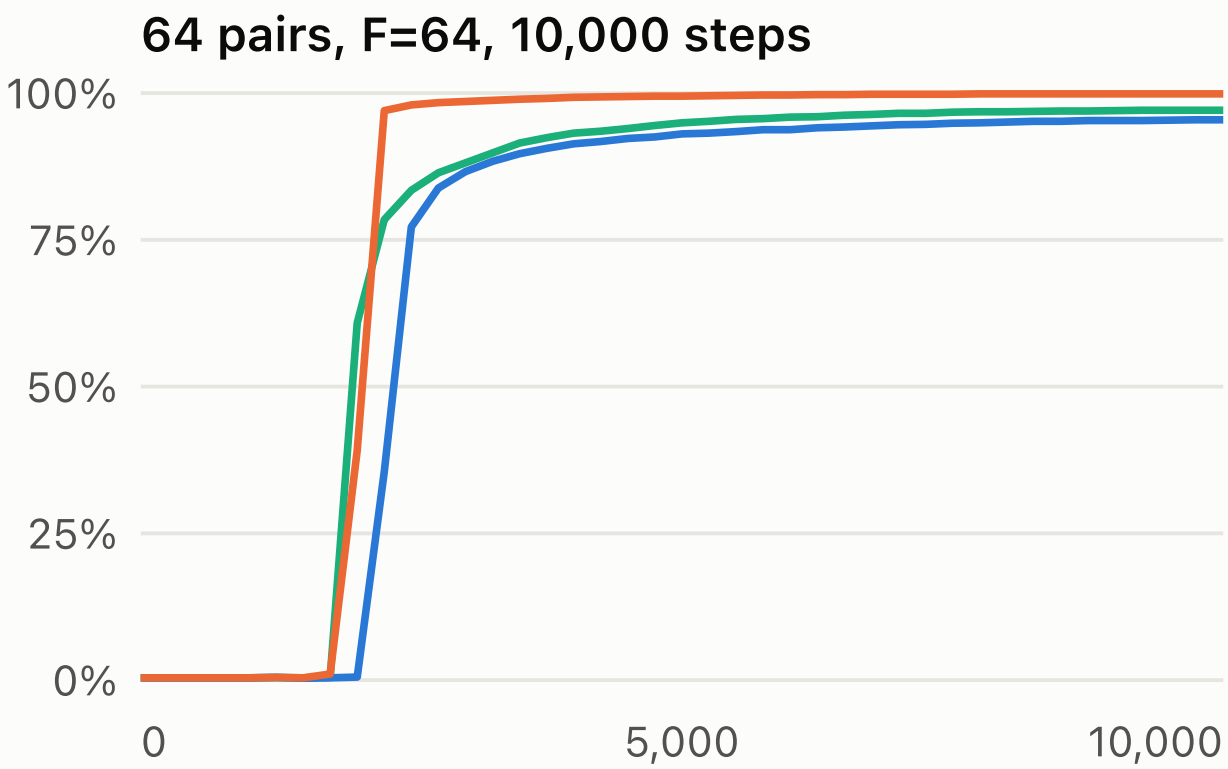
drawers, hard 4×6b · drawers, hard 16×4b · drawers, hard 4×4b · linear attention (reference, dotted) · linear attention · drawers + neighbour 4×6b · drawers + neighbour 4×4b · drawers + neighbour 16×4b · seed 1 (dashed)


32 pairs, F=64
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=256
100%
75%
50%
25%
0%
0
1,500
3,000


64 pairs, F=64
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=256, 10,000 steps
100%
75%
50%
25%
0%
0
5,000
10,000



▼ Run table

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
32 pairs, F=256	drawers + neighbour 16×4b	0	99.75%	1000	526,852	17.0	81.7
32 pairs, F=256	drawers + neighbour 16×4b	1	99.73%	1000	526,852	17.0	95.4
32 pairs, F=256	drawers + neighbour 4×6b	0	99.84%	2000	526,852	10.7	82.9
32 pairs, F=256	drawers + neighbour 4×6b	1	99.94%	2000	526,852	10.7	81.8
32 pairs, F=256	drawers, hard 16×4b	0	99.81%	1000	526,850	14.8	82.6
32 pairs, F=256	drawers, hard 16×4b	1	99.84%	1250	526,850	14.8	81.2
32 pairs, F=256, 10,000 steps	drawers, hard 4×6b	0	100.00%	2750	526,850	30.0	89.7

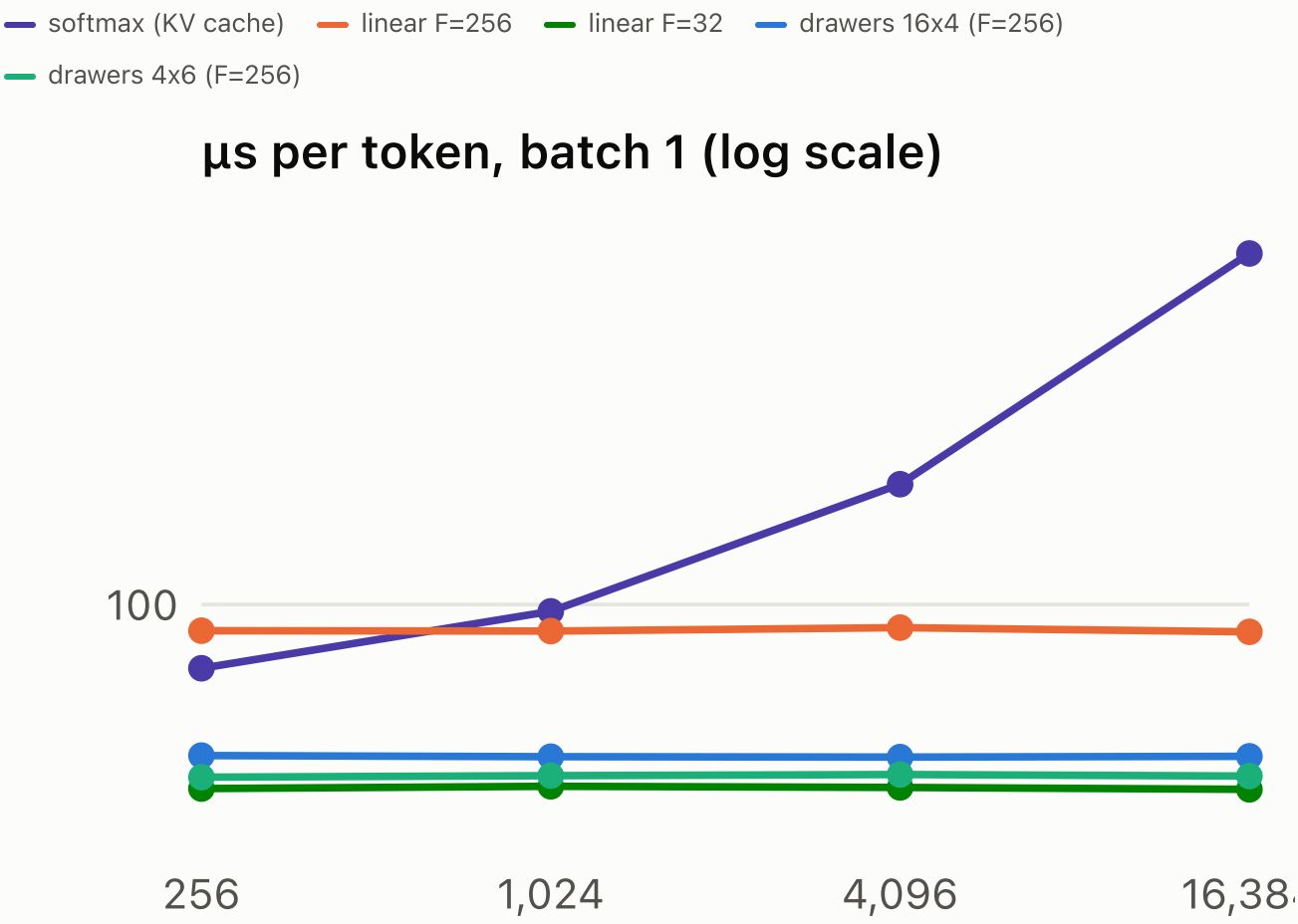
setting	model	seed	final recall	step reaching 50%	params	train min	max °C
32 pairs, F=64	drawers + neighbour 4×4b	0	96.32%	1250	526,852	6.9	68.8
32 pairs, F=64	drawers + neighbour 4×4b	1	96.32%	1250	526,852	7.0	72.3
64 pairs, F=64	drawers + neighbour 4×4b	0	64.94%	2250	526,852	14.2	90.0
64 pairs, F=64, 10,000 steps	drawers + neighbour 4×4b	0	97.09%	2000	526,852	47.2	91.4
64 pairs, F=64, 10,000 steps	drawers, hard 4×4b	0	95.44%	2500	526,850	43.7	92.9
64 pairs, F=64, 10,000 steps	linear attention	0	99.87%	2250	625,152	28.8	92.2

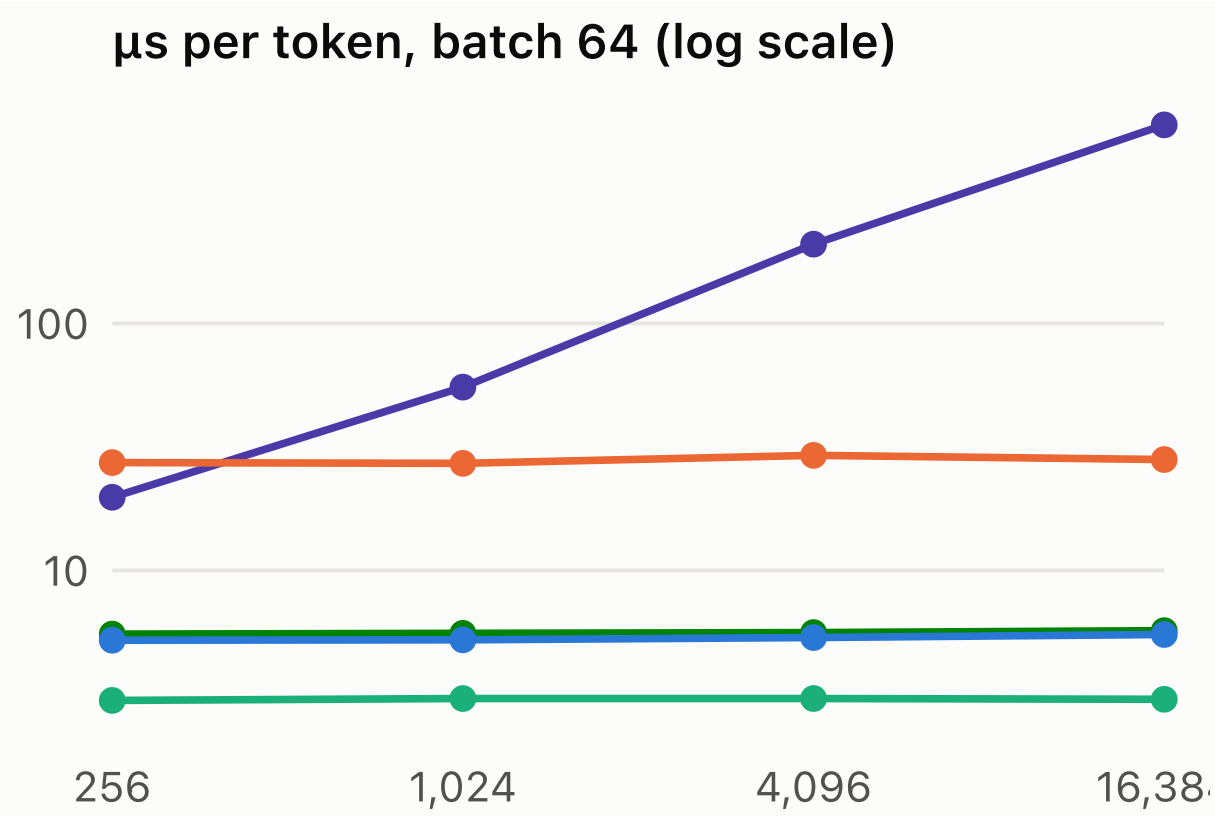
Decode speed benchmark

Question. Measured, not calculated: how long does one attention-replacement layer take per new token, as the context grows from 256 to 16,384 tokens? CPU, PyTorch, recurrent (step-by-step) form; the drawer step was checked to reproduce the training-time output to 1e-6.

What we learned. Measured, at 64 sequences per batch (throughput): drawers 16×4 cost 5.2–5.7 µs per token and 4×6 cost 3.0 µs (but 4×6 learns unreliably), against 27–29.5 µs for linear attention with the same state (F=256): **5× and 9× faster at equal memory**, flat in context length. Against linear attention with ~equal parameters (F=32, 8× smaller state) drawers 16×4 are equally fast. Softmax with a KV cache costs 20 µs per token at 256 tokens and 635 µs at 16,384 (115× drawers 16×4), with 16 MB of state per sequence instead of 132 KB. At batch 1 framework overhead dominates: drawers 40–44 µs, linear F=256 87 µs, linear F=32 37 µs. The analytic estimate was ~60× less arithmetic than linear F=256; the measured gain is 5–9×, because projections and gather/scatter overhead do not shrink with sparsity (PyTorch CPU, no custom kernel). Memory-size sweep (context 1,024): linear F=32/64/128/256 cost 5.2/7.8/13.7/28.2 µs per token; drawers with 4-bit tables cost 2.9 (F=64), 4.0 (F=128), 5.4 (F=256): the gap grows with memory size because drawers touch a fixed number of rows. Bounded soft reads as implemented (score every cell of a table, then take the top 4): 16 × 4 top-4 costs 31.1 µs per token, more than linear attention (28.3); 4 × 6 top-4 costs 13.0 µs, 2.2× cheaper than linear F=256 but 2.3× the

hard design (5.7). Choosing the 4 cells directly from the 2 least-certain bits (flip-2) brings this down only to 10.9 µs: most of the extra cost is the weighted reads and writes of 4 rows per table and PyTorch's per-operation overhead, which a fused kernel should reduce. Cheap flip-2 soft read (4 × 6 bits): 10.9 µs per token (exact top-4: 13.2; hard 16 × 4: 5.5; linear F=256: 29.5). Timings vary by about ±5% between benchmark runs.





▶ Benchmark table

Cost by memory size (batch 64, context 1,024)

mixer	memory F	µs/token	state KB/seq
linear F=32	32	6.95	16.5
linear F=64	64	9.32	33.0
linear F=128	128	15.97	66.0
linear F=256	256	29.45	132.0
drawers 4x4 (F=64)	64	2.88	33.0
drawers 8x4 (F=128)	128	4.31	66.0
drawers 16x4 (F=256)	256	5.45	132.0
drawers 16x2 (F=64)	64	5.05	33.0
drawers 64x2 (F=256)	256	16.9	132.0
drawers 4x6 (F=256)	256	3.14	132.0
drawers 16x4 top-4 soft (F=256)	256	30.06	132.0
drawers 4x6 top-4 soft (F=256)	256	13.24	132.0

mixer	memory F	µs/token	state KB/seq
drawers 4x6 flip-2 soft (F=256)	256	10.94	132.0
drawers 16x4 flip-2 soft (F=256)	256	32.1	132.0

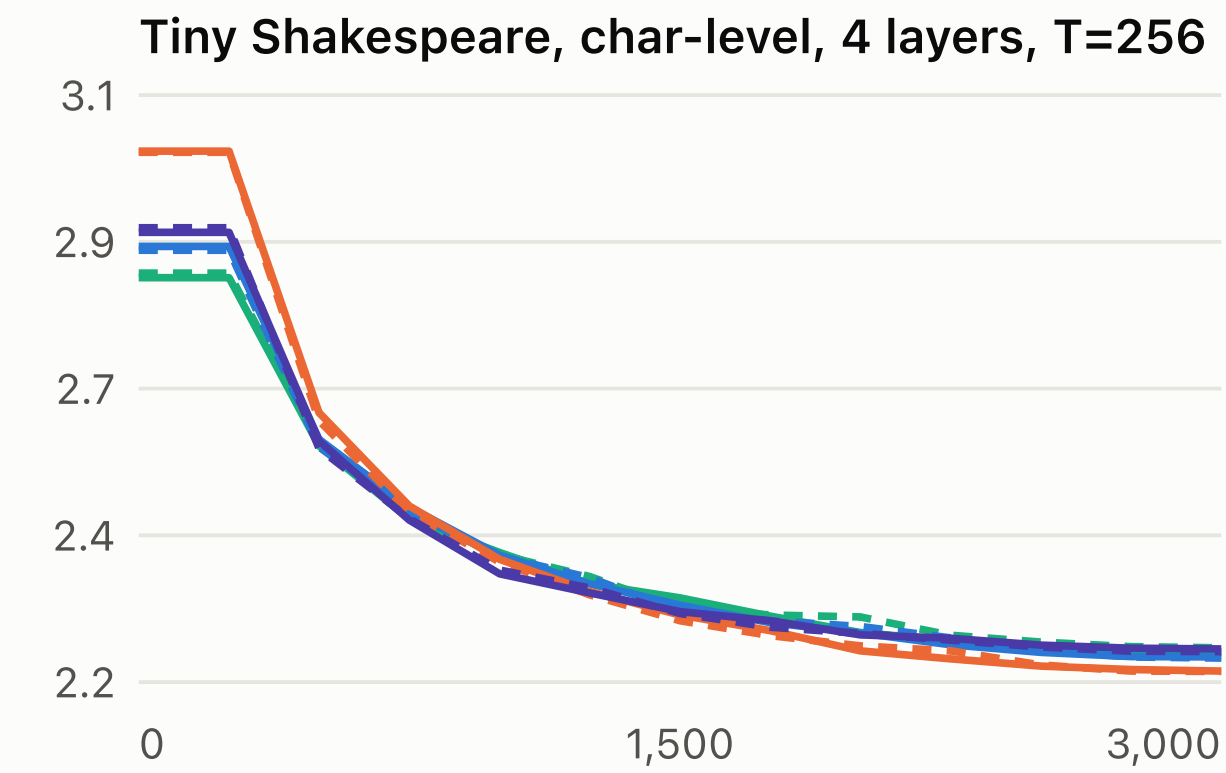
Batch 5 — proofs of concept: real text, gaps, 64 pairs

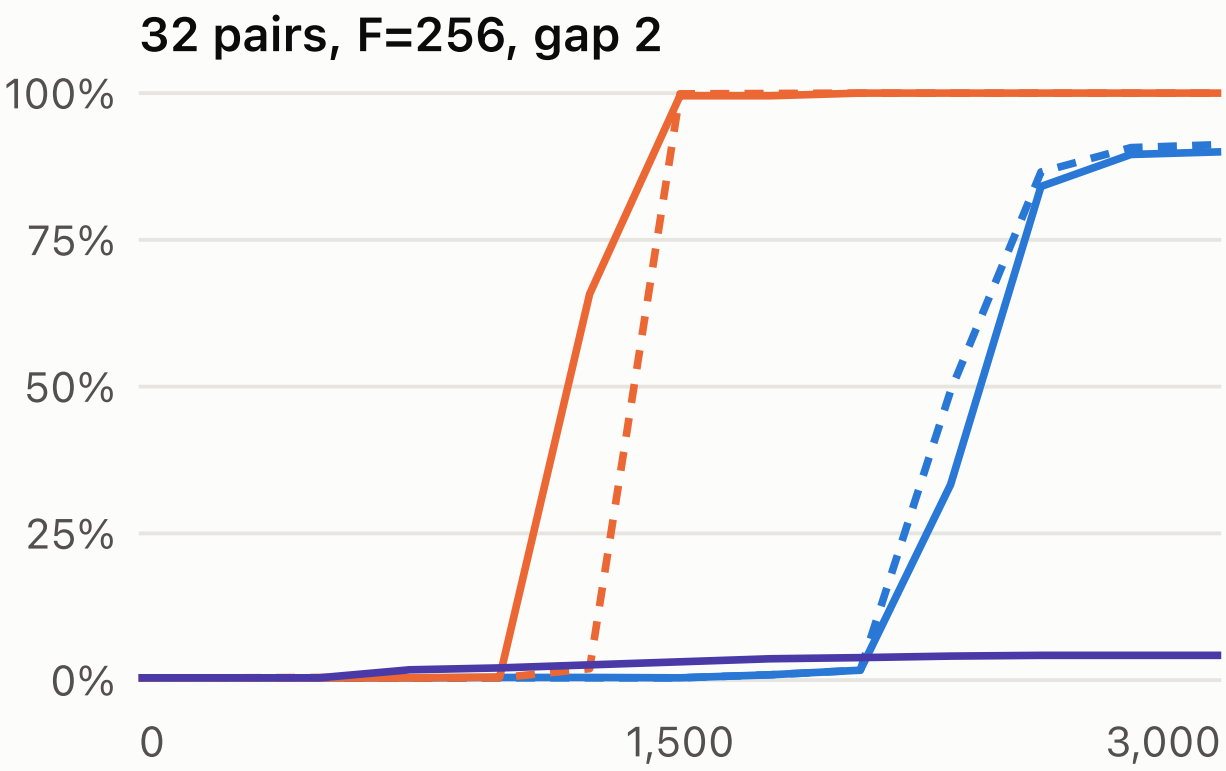
17 of 17 runs finished · 2026-10-19, finished 20:33

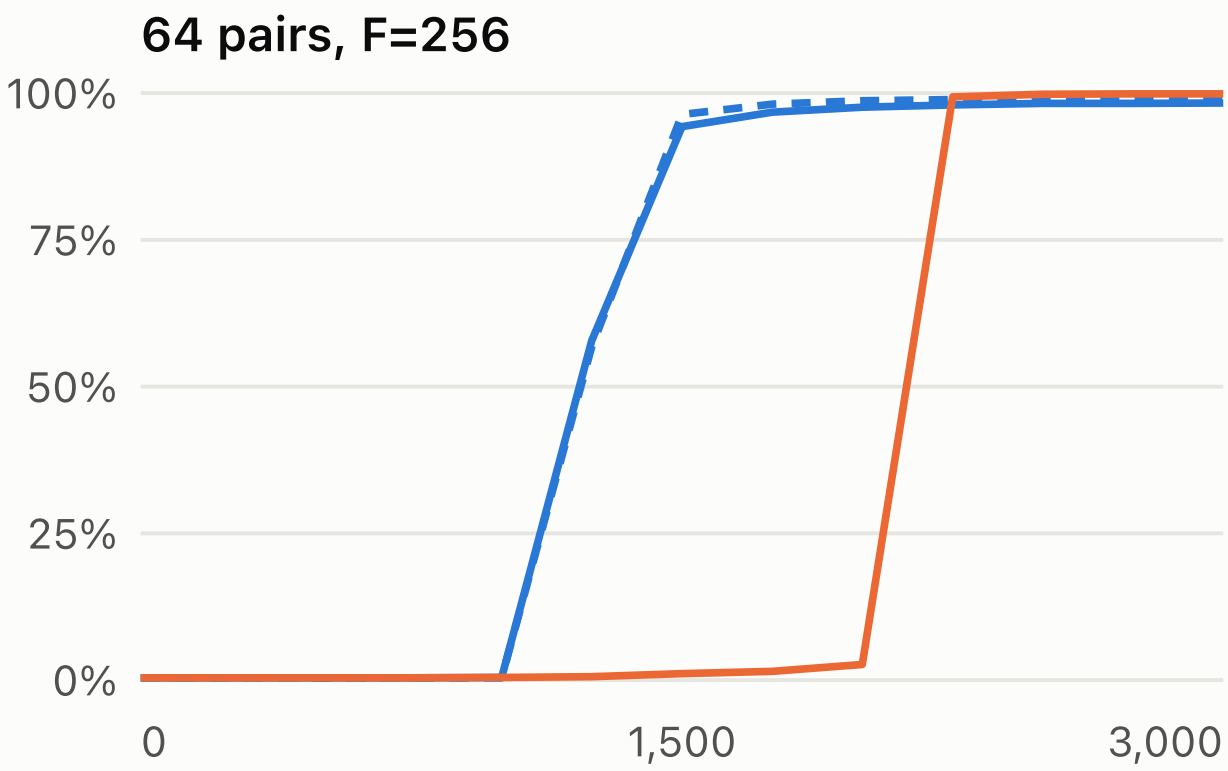
Question. Does drawer memory work on real text (character-level Shakespeare)? Does recall survive filler tokens between key and value, and 64 pairs with small tables? Anchors are drawn per seed from here on.

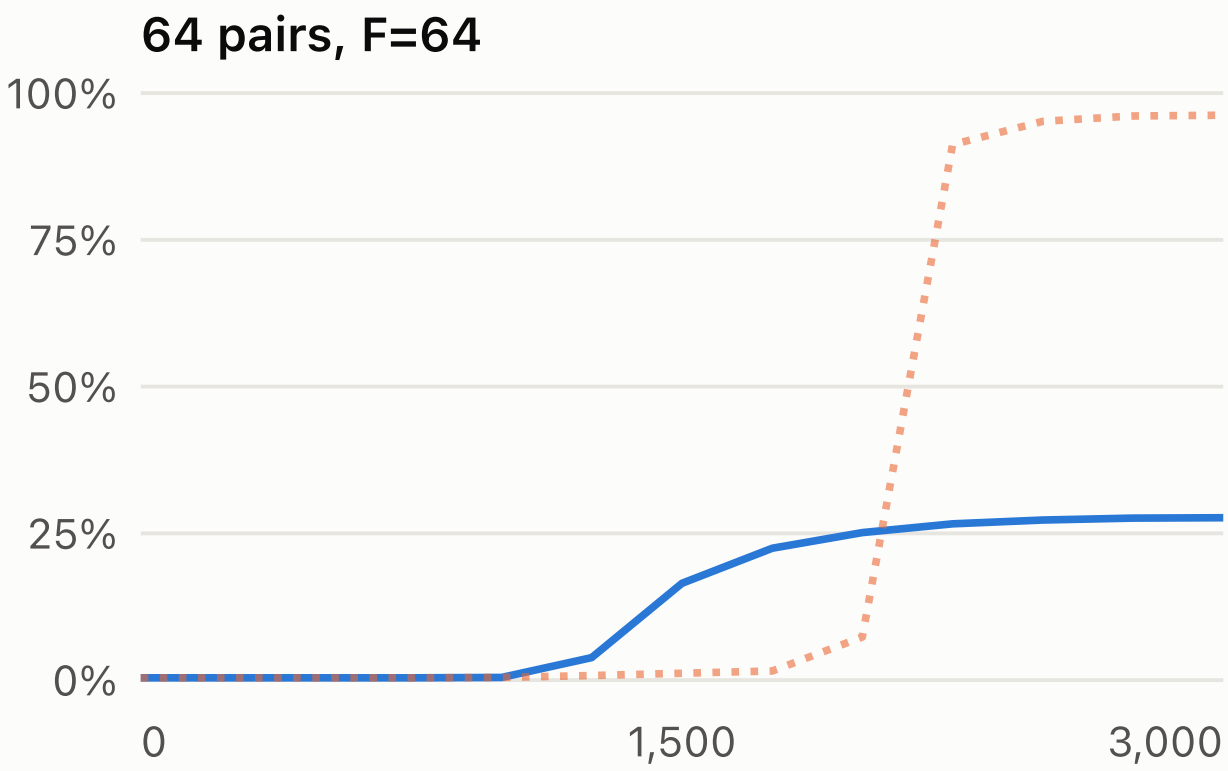
What we learned. On Tiny Shakespeare (two seeds) drawers 16×4 reach 2.237–2.239 bpc, between linear attention (2.217) and softmax (2.247–2.251); seed spread ≤0.004, so the ordering is real. Neighbour cell: 2.246 / 2.253, a faster start but a slightly worse end, so it does not help on text with 4-bit tables. Drawers improve from the first evaluation, without the delayed 'click' seen on recall. All models see only local order through the 4-character token shift, which handicaps softmax relative to normal practice. Recall with 2 filler tokens between key and value (two seeds): linear attention 99.99–99.997%, drawers 16×4 90.0% / 91.2%; drawers start learning only at step ~2,250 (no gap: ~1,000), so the gap mainly slows the store/read agreement down. Softmax attention (no positional encoding, seed 0) did not learn the gapped task within 3,000 steps (4.3%), so drawers beat softmax but trail linear attention here. 64 pairs, memory 256 (two seeds): drawers 16×4 learn from step ~1,250 (linear attention ~2,250) and reach 98.3% / 99.1% vs 99.9%. At the other extreme, 16 tables × 2 bits (4 cells each, memory 64) at 64 pairs starts learning early (step ~1,500) but plateaus at 27.7%: with 4 cells per table every cell holds ~16 of the 64 pairs, a hard collision ceiling. The bits-per-table trade-off has two sides: too many bits slow learning, too few cap capacity; 4 bits was the sweet spot at both 32 and 64 pairs. Caveat found later: the runs make ~25 passes over Tiny Shakespeare and all models overfit (final training bpc: softmax 1.75, linear 1.79, drawers 1.83, half-width hybrid 1.64 vs validation 2.22–2.30). Drawers overfit least, which helps their validation score.

drawers, hard 16×4b · drawers, hard 16×2b · linear attention (reference, dotted)
linear attention · drawers + neighbour 16×4b · softmax attention · seed 1 (dashed)


Tiny Shakespeare, char-level, 4 layers, T=256
3.1
2.9
2.7
2.4
2.2
0
1,500
3,000


32 pairs, F=256, gap 2
100%
75%
50%
25%
0%
0
1,500
3,000





▼ Run table

setting	model	seed	final val bpc	best	params	train min	max °C
32 pairs, F=256, gap 2	drawers, hard 16×4b	0	0.900	0.900	629,250	25.7	86.9

setting	model	seed	final val bpc	best	params	train min	max °C
32 pairs, F=256, gap 2	drawers, hard 16×4b	1	0.912	0.912	629,250	24.1	84.3
32 pairs, F=256, gap 2	linear attention	0	1.000	1.000	1,120,768	15.7	82.8
32 pairs, F=256, gap 2	linear attention	1	1.000	1.000	1,120,768	15.4	82.2
32 pairs, F=256, gap 2	softmax attention	0	0.043	0.043	662,016	6.8	84.4
64 pairs, F=256	drawers, hard 16×4b	0	0.983	0.983	526,850	31.9	85.6
64 pairs, F=256	drawers, hard 16×4b	1	0.991	0.991	526,850	35.2	83.2
64 pairs, F=256	linear attention	0	0.999	0.999	1,018,368	20.0	83.5
64 pairs, F=64	drawers, hard 16×2b	0	0.277	0.277	526,850	18.4	85.4
Tiny Shakespeare, char-level, 4 layers, T=256	drawers + neighbour 16×4b	0	2.246	2.244	1,004,552	35.2	91.3
Tiny Shakespeare, char-level, 4 layers, T=256	drawers + neighbour 16×4b	1	2.253	2.253	1,004,552	36.2	84.2
Tiny Shakespeare, char-level, 4 layers, T=256	drawers, hard 16×4b	0	2.239	2.239	1,004,548	28.0	92.1
Tiny Shakespeare, char-level, 4 layers, T=256	drawers, hard 16×4b	1	2.237	2.237	1,004,548	31.1	83.5
Tiny Shakespeare, char-level, 4 layers, T=256	linear attention	0	2.217	2.217	1,987,584	16.2	93.8
Tiny Shakespeare, char-level, 4 layers, T=256	linear attention	1	2.216	2.216	1,987,584	21.0	83.7
Tiny Shakespeare, char-level, 4 layers, T=256	softmax attention	0	2.251	2.251	1,070,080	7.8	89.0
Tiny Shakespeare, char-level, 4 layers, T=256	softmax attention	1	2.247	2.247	1,070,080	10.6	86.2

▶ Generated text samples (seed 0, temperature 0.8)

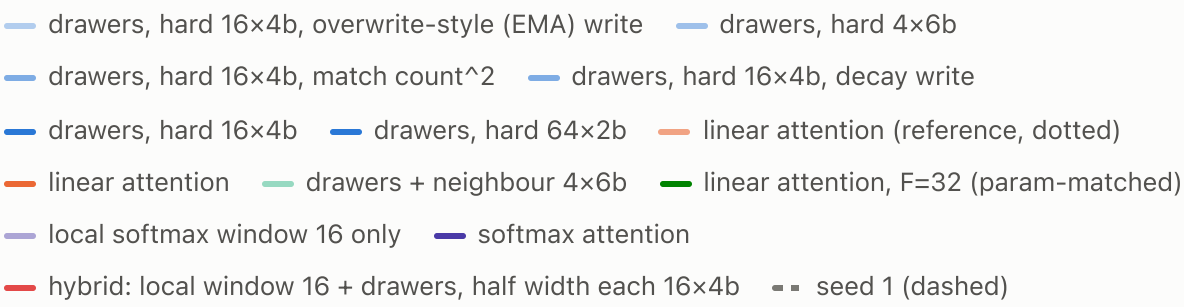
Batch 6 — diagnostics, memory editing, hybrids

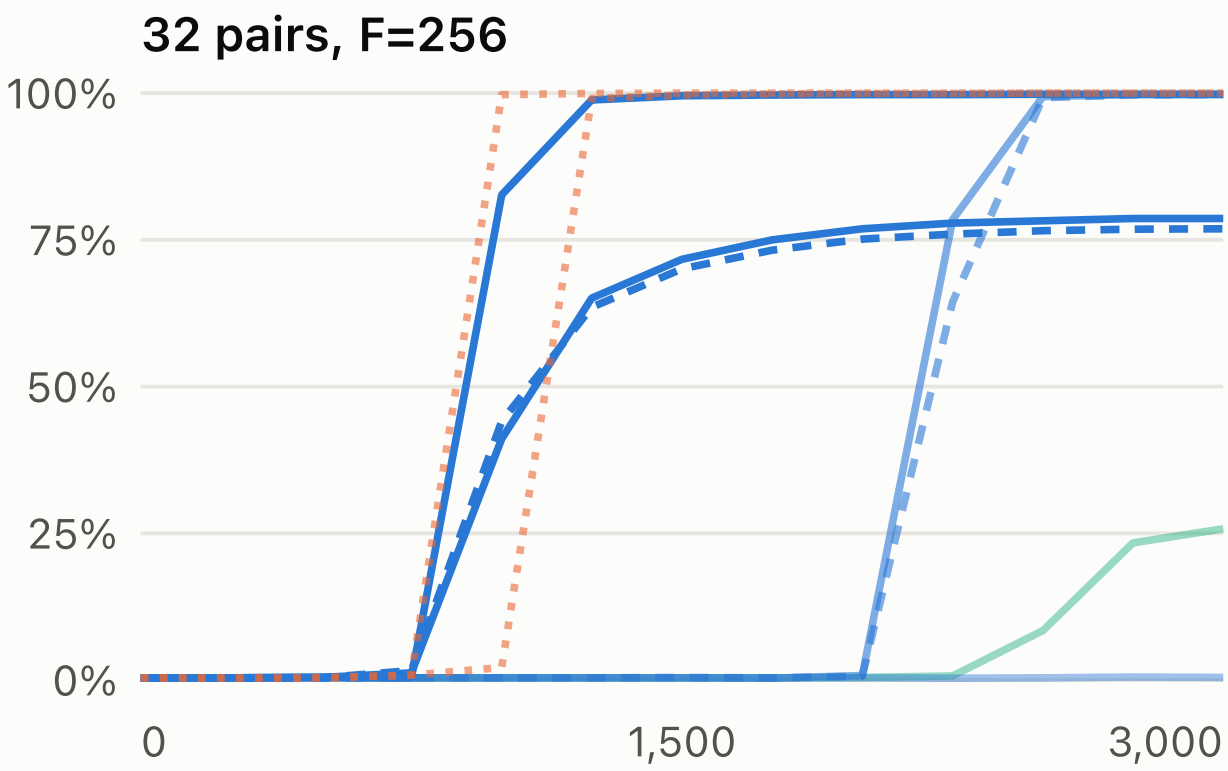
24 of 24 runs finished · 2026-10-19 evening – 2026-10-20 01:49

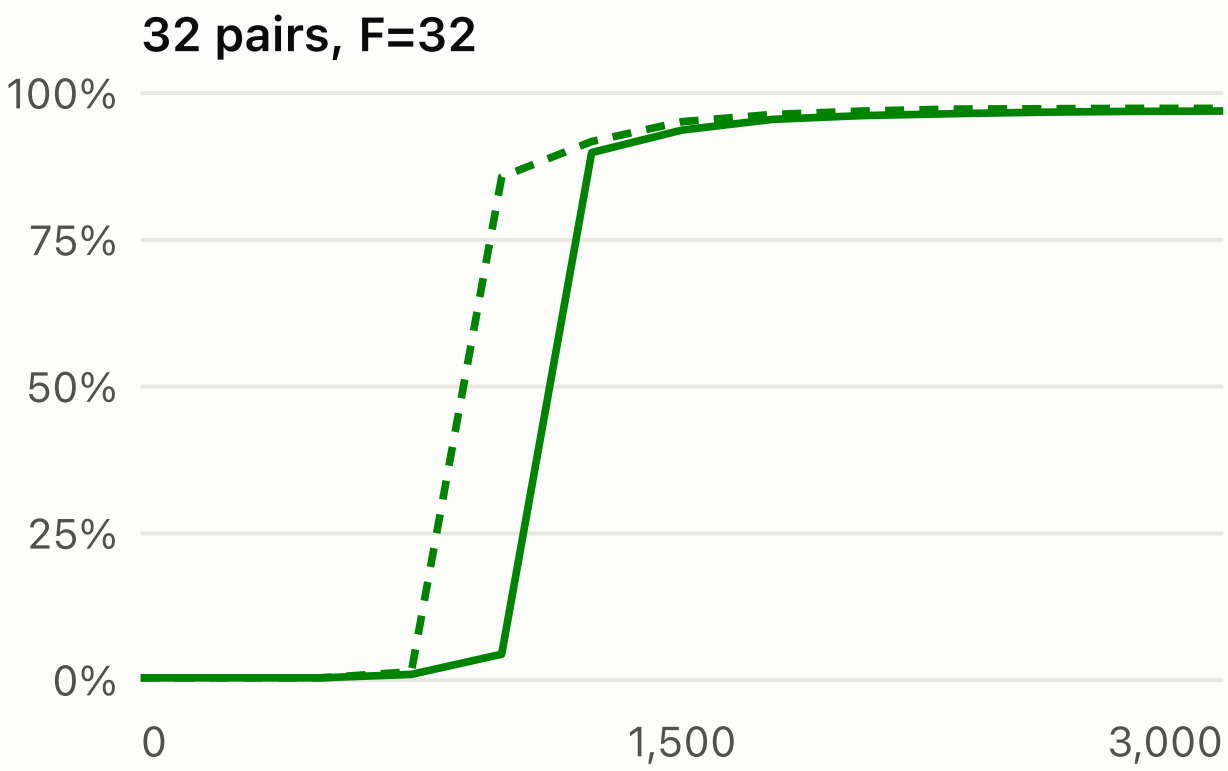
Question. Why does learning click when it does (address-match rate)? Can the memory replace old values? Does a local attention window plus drawers beat either alone? How does accuracy trade against cells touched?

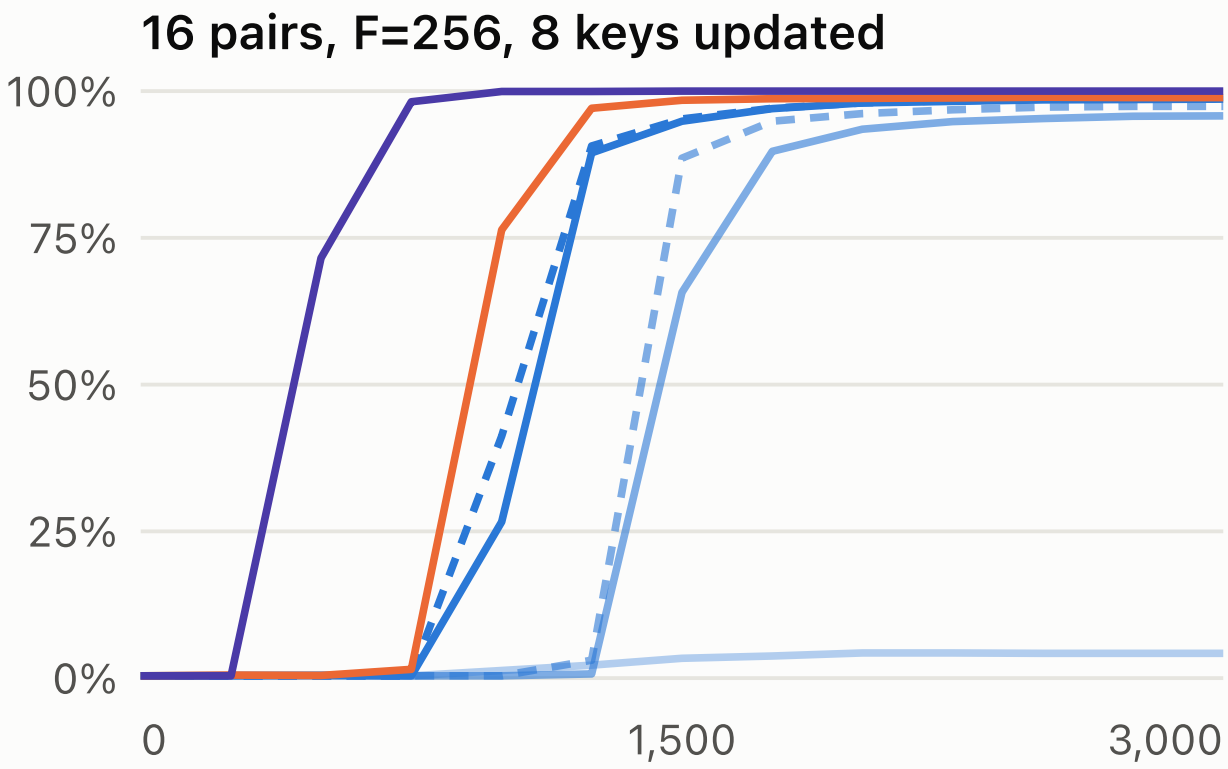
What we learned. Address diagnostics (16 tables × 4 bits, 32 pairs): the recall click coincides with a jump of layer-1 store/read address agreement from 19% to 81% of tables between steps 750 and 1,000 (final 93%). Hashing quality is good from the start (entropy ≥0.93 of maximum, bit bias ≤0.1, unrelated-pair collisions ≈ 1/16 = chance), so learning is limited by agreement, not by address quality. Layer 2's addresses are collapsed into ~13% of cells before the click (entropy 0.09) and spread out afterwards; layer 2 does not do key matching (agreement 40%). The correct value's share of the layer-1 read is ~11% (own write and the key token's write share the same cells). Failed run (4 tables × 6 bits, same task, 0.44% recall): layer-1 agreement crawls from 2% to 11% over 3,000 steps and only 32% of queries match their value in at least one table. The bits become correlated (mean |corr| 0.09 → 0.31) and entropy falls from 0.97 to 0.80: the optimiser is effectively merging bits (fewer distinct cells) to raise agreement, which quadruples unrelated-pair collisions (0.067 vs chance 0.016) without crossing the threshold. This is the mechanism behind 'fewer bits per table learn faster'. With the neighbour cell (4 × 6 bits, anchors drawn for this seed) agreement crawls the same way to 11% at step 2,000, then crosses: 46% at 2,500, 56% at 3,000 (recall 25.8% and rising when training ended). After the crossing the bits de-correlate again (|corr| 0.18 → 0.07) and collisions fall back to chance. With the original anchors the same variant had reached 99.8–99.9%: 6-bit tables are sensitive to the anchor draw, 4-bit tables have not been. Parameter-matched baseline: linear attention with F=32 (559,616 parameters, vs 526,850 for drawers) reaches 96.9% / 97.4% at 32 pairs (two seeds), below drawers 16×4 (99.8%) at the same measured decode cost (5.5 vs 5.2 µs per token). Memory editing (16 pairs, 8 keys re-assigned; seed 0): softmax 99.99% on updated keys / 100% on unchanged; linear attention 98.0% / 99.9%; drawers 16×4 with additive writes 98.0% / 99.3% (seed 1: 98.2% / 99.3%). Despite averaging old and new values in shared cells, both kernel models learned to favour the newer value, probably using order information carried by the token shift. Decay writes (learned per-head forgetting) are worse, not better: 93.8% / 96.5% on updated keys and 97.7% / 98.3% on unchanged ones (two seeds), with a later click (step ~1,500): forgetting also erodes the older facts that are still needed. Overwrite-style (EMA) writes failed (seed 0): 4.2% overall, never clicking. With 16-cell tables, ~5 of the 80 tokens share each cell, and every token writes, so each write partly replaces what the earlier tokens stored; the learned per-token

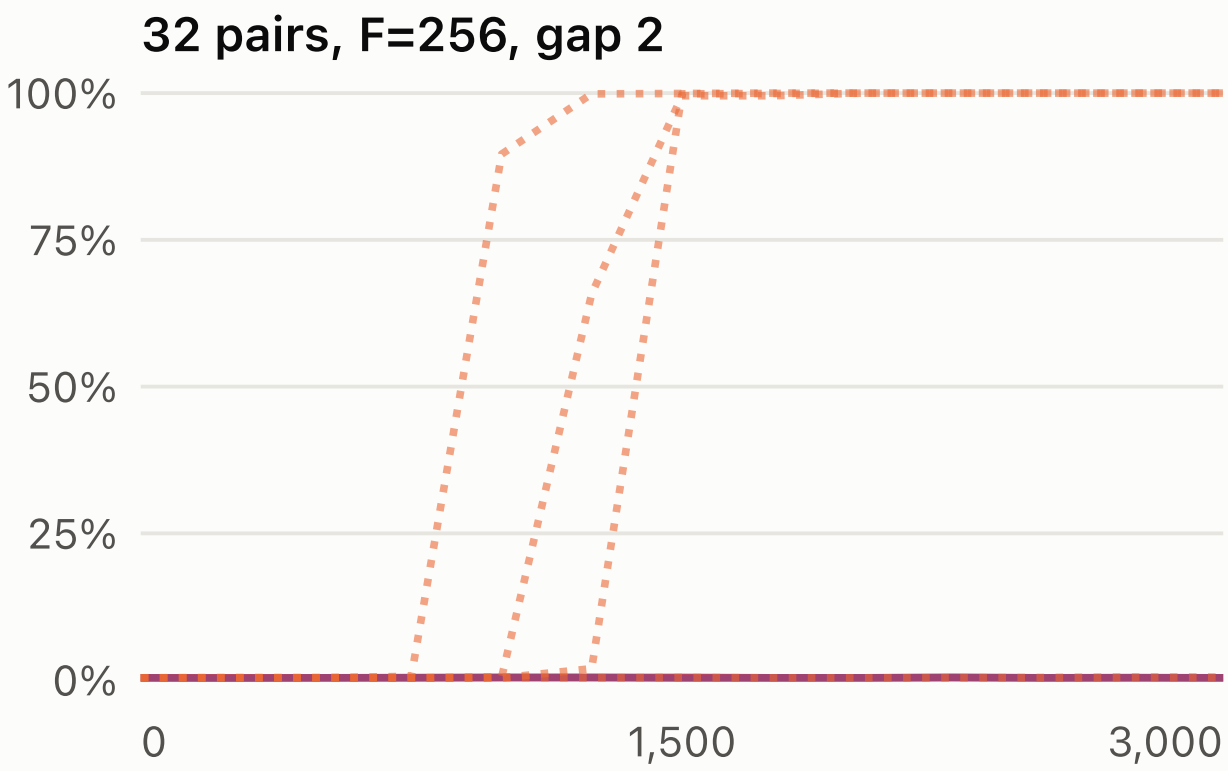
write rate would have to learn to stay near zero for all but value tokens before any retrieval works. Selective writing must come before overwriting. (The second seed of the overwrite-style write was skipped deliberately: the failure is structural, not seed luck.) Cells touched vs accuracy (32 pairs, memory 256, seed 0): 64 tables × 2 bits (64 cells touched per token) learn early (41% at step 1,000) but plateau at 78.7% / 76.9% (two seeds), against 99.8% for 16 tables × 4 bits (16 cells touched): tiny tables are both less accurate and 4× more memory accesses. Soft intersection (read weight = shared-table count squared, 16 × 4, two seeds): 99.75% / 99.74% final, no better than the plain sum (99.8%), and it clicked late on both seeds (step ~2,250 vs ~1,000): sharpening the match makes the start harder, like adding bits. Plain summed reads remain the best choice. Hybrid with the width split in half (2 heads local softmax window 16 + 2 heads drawers, same total size) did not learn the gapped recall task at all (0.4%, loss flat at ln 256): halving the drawer part (2 heads, half the features) removed the little margin drawers had on this task. A layer-wise hybrid (layer 1 local window, layer 2 full-width drawers) was added to batch 7. Control: a local softmax window of 16 alone stays at chance on gapped recall (0.39%), as expected: answers lie 32–190 tokens back, beyond the window. On Tiny Shakespeare the half-width hybrid learns fastest of all models early (2.333 bpc at step 1,000 vs 2.366–2.395) but stalls after step ~1,500 and ends at 2.298 / 2.288 (two seeds), worse than plain drawers (2.237–2.239); validation loss rises slightly at the end while training loss keeps falling. Local window only (16 characters, no long-range mixer) on Tiny Shakespeare: 2.468 / 2.474 bpc on held-out text (two seeds; training 1.46–1.47, the strongest overfitting), far worse than drawers (2.24): the long-range memory is doing real work on text, worth ~0.23 bits per character here. Parameter-matched linear attention (F=32) on Tiny Shakespeare: 2.225 bpc (seed 0), slightly better than drawers 16×4 (2.237–2.239): the recall advantage at equal parameters does not carry over to this text task.

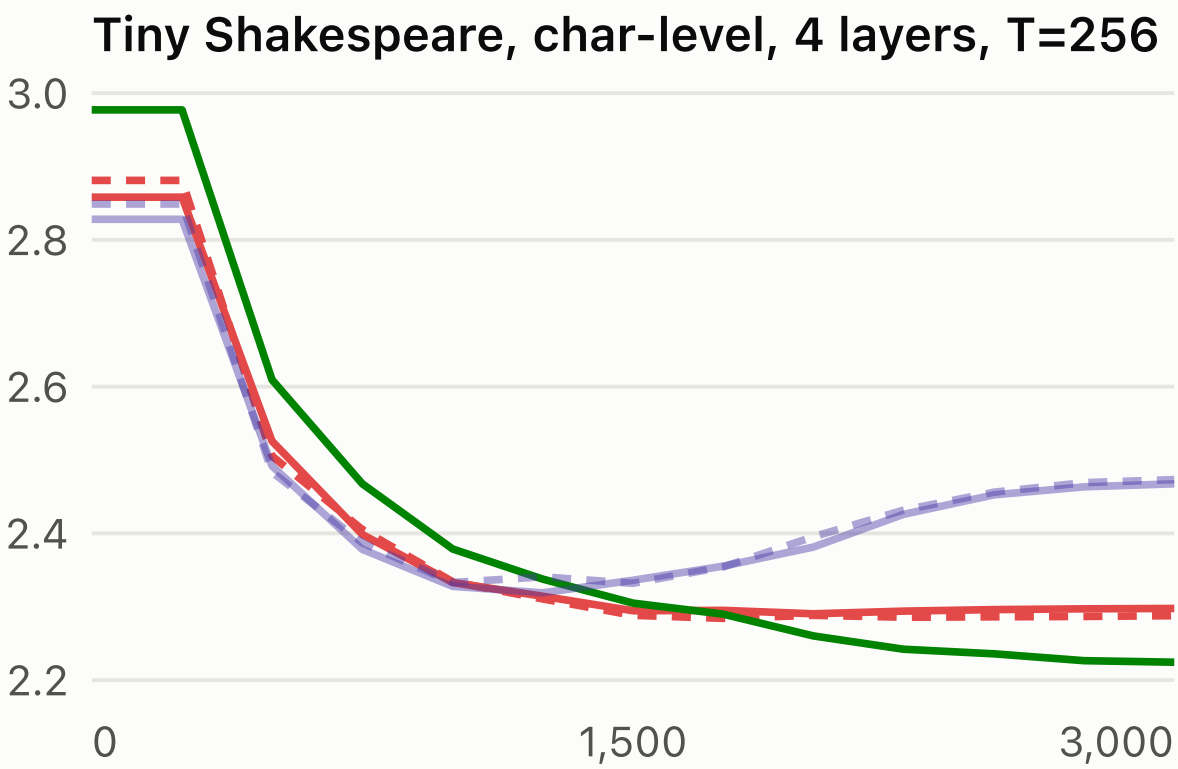



32 pairs, F=256
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=32
100%
75%
50%
25%
0%
0
1,500
3,000


16 pairs, F=256, 8 keys updated
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=256, gap 2
100%
75%
50%
25%
0%
0
1,500
3,000



▼ Run table

setting	model	seed	final recall	step reaching 50%	updated keys	unchanged keys	params	train min	max °C
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b	0	98.66 %	1250	98.0%	99.3%	526,850	9.3	80.9
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b	1	98.73 %	1250	98.2%	99.3%	526,850	9.3	82.3
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b, decay write	0	95.77 %	1500	93.8%	97.7%	526,858	9.4	81.5
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b, decay write	1	97.42 %	1500	96.5%	98.3%	526,858	9.4	82.1

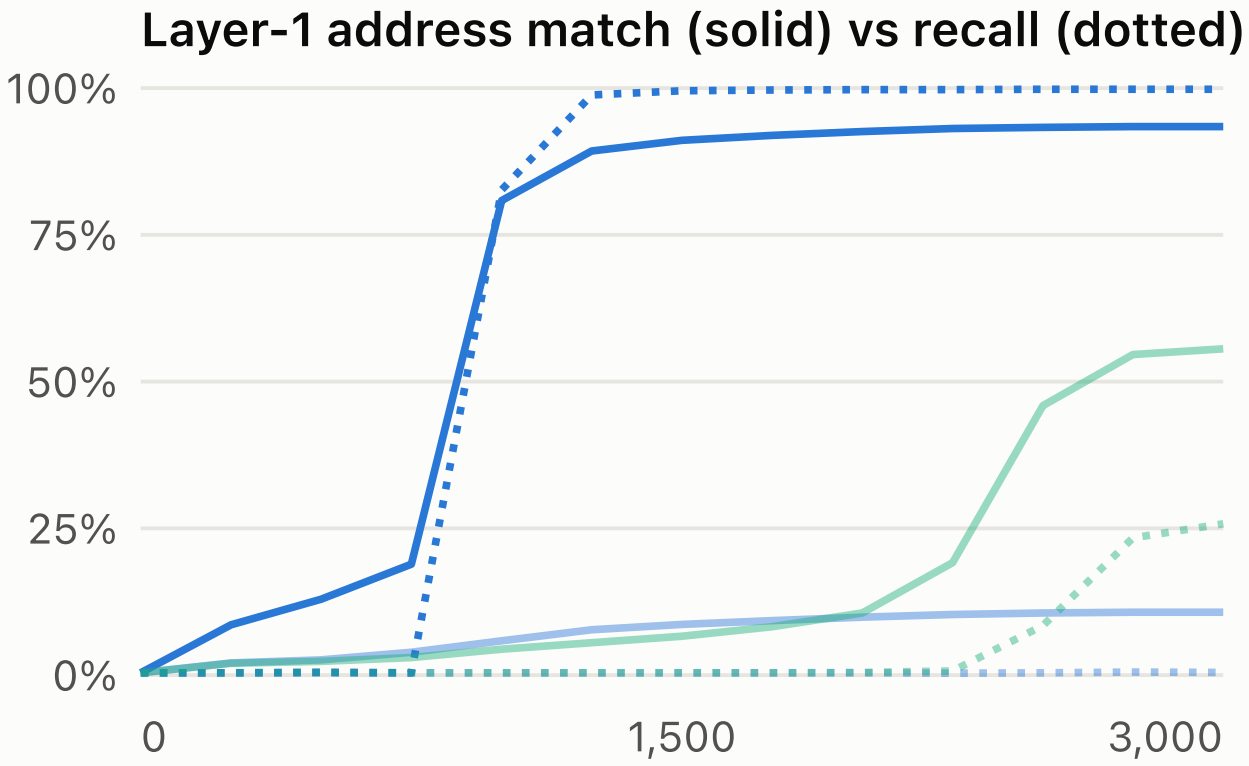
setting	model	seed	final recall	step reaching 50%	updated keys	unchanged keys	params	train min	max °C
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b, overwrite-style (EMA) write	0	4.19 %	never	4.6%	3.8%	527,882	40.8	85.3
16 pairs, F=256, 8 keys updated	drawers, hard 16×4b, overwrite-style (EMA) write	1	skipped (seed 0 failed)						
16 pairs, F=256, 8 keys updated	linear attention	0	98.92 %	1000	98.0%	99.9%	1,018,368	5.9	83.0
16 pairs, F=256, 8 keys updated	softmax attention	0	99.99 %	500	100.0%	100.0%	559,616	2.1	83.3
32 pairs, F=256	drawers + neighbour 4×6b	0	25.81 %	never			526,852	11.5	84.2
32 pairs, F=256	drawers, hard 16×4b	0	99.80 %	1000			526,850	15.0	82.7
32 pairs, F=256	drawers, hard 16×4b, match count^2	0	99.75 %	2250			526,850	15.8	84.7
32 pairs, F=256	drawers, hard 16×4b,	1	99.74 %	2250			526,850	15.8	85.9

setting	model	seed	final recall	step reaching 50%	updated keys	unchanged keys	params	train min	max °C
	match count^2								
32 pairs, F=256	drawers, hard 4×6b	0	0.44 %	never			526,850	9.5	83.3
32 pairs, F=256	drawers, hard 64×2b	0	78.65 %	1250			526,850	22.3	86.3
32 pairs, F=256	drawers, hard 64×2b	1	76.90 %	1250			526,850	22.9	89.0
32 pairs, F=256, gap 2	hybrid: local window 16 + drawers, half width each 16×4b	0	0.42 %	never			604,674	14.9	83.1
32 pairs, F=256, gap 2	local softmax window 16 only	0	0.39 %	never			662,016	6.6	85.5
32 pairs, F=32	linear attention, F=32 (param-matched)	0	96.92 %	1250			559,616	4.0	84.8
32 pairs, F=32	linear attention, F=32 (param-matched)	1	97.43 %	1000			559,616	3.8	82.8
Tiny Shakesp	hybrid: local	0	229.75%	250			955,396	19.9	83.6

setting	model	seed	final recall	step reaching 50%	updated keys	unchanged keys	params	train min	max °C
eare, char-level, 4 layers, T=256	window 16 + drawers, half width each 16×4b								
Tiny Shakespeare, char-level, 4 layers, T=256	hybrid: local window 16 + drawers, half width each 16×4b	1	228.76%	250			955,396	19.8	82.9
Tiny Shakespeare, char-level, 4 layers, T=256	linear attention, F=32 (param-matched)	0	222.45%	250			1,070,080	9.7	82.2
Tiny Shakespeare, char-level, 4 layers, T=256	local softmax window 16 only	0	246.77%	250			1,070,080	8.9	82.3
Tiny Shakespeare, char-level, 4 layers, T=256	local softmax window 16 only	1	247.35%	250			1,070,080	9.0	84.6

Address diagnostics

Measured on 256 fresh sequences. match = query lands in the same cell as the token that stored its value (share of tables); collide = shares a cell with an unrelated pair (chance = 1/cells); read share = weight of the correct value in what the read returns.



model	layer	match	≥1 table	collide (chance)	occupancy	entropy	bit bias	bit corr	read share
drawers, hard 16×4b	1	0.935	1.000	0.062 (0.062)	0.900	0.993	0.010	0.049	0.113
drawers, hard 16×4b	2	0.400	0.994	0.107 (0.062)	0.920	0.988	0.013	0.068	0.036
drawers, hard 16×4b	1	0.935	1.000	0.062 (0.062)	0.900	0.993	0.010	0.049	0.113
drawers, hard 16×4b	2	0.400	0.994	0.107 (0.062)	0.920	0.988	0.013	0.068	0.036
drawers, hard 4×6b	1	0.107	0.318	0.067 (0.016)	0.500	0.805	0.072	0.307	0.015
drawers, hard 4×6b	2	0.670	0.990	0.641 (0.016)	0.078	0.184	0.432	0.222	0.011
drawers, hard 4×6b	1	0.107	0.318	0.067 (0.016)	0.500	0.805	0.072	0.307	0.015
drawers, hard 4×6b	2	0.670	0.990	0.641 (0.016)	0.078	0.184	0.432	0.222	0.011

model	layer	match	≥1 table	collide (chance)	occupancy	entropy	bit bias	bit corr	read share
drawers + neighbour 4×6b	1	0.556	0.948	0.017 (0.016)	0.578	0.974	0.021	0.073	0.158
drawers + neighbour 4×6b	2	0.344	0.705	0.151 (0.016)	0.262	0.568	0.254	0.283	0.025
drawers + neighbour 4×6b	1	0.556	0.948	0.017 (0.016)	0.578	0.974	0.021	0.073	0.158
drawers + neighbour 4×6b	2	0.344	0.705	0.151 (0.016)	0.262	0.568	0.254	0.283	0.025

▶ Generated text samples (seed 0, temperature 0.8)

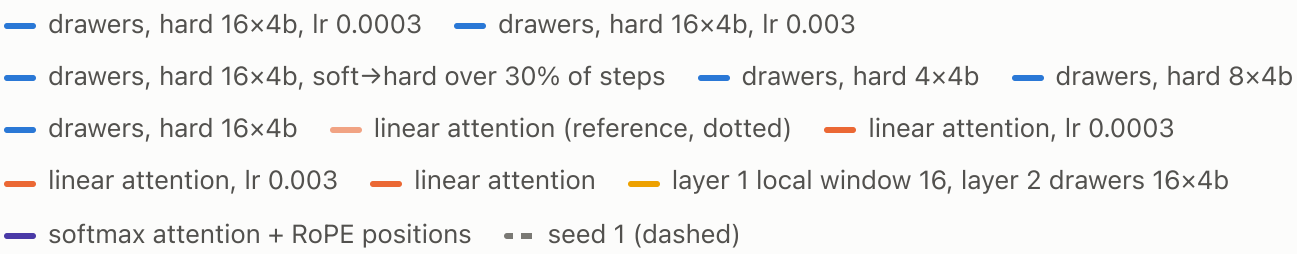
Batch 7 — layer-wise hybrid, fair softmax, soft-to-hard addressing, accuracy vs cost, scale-up

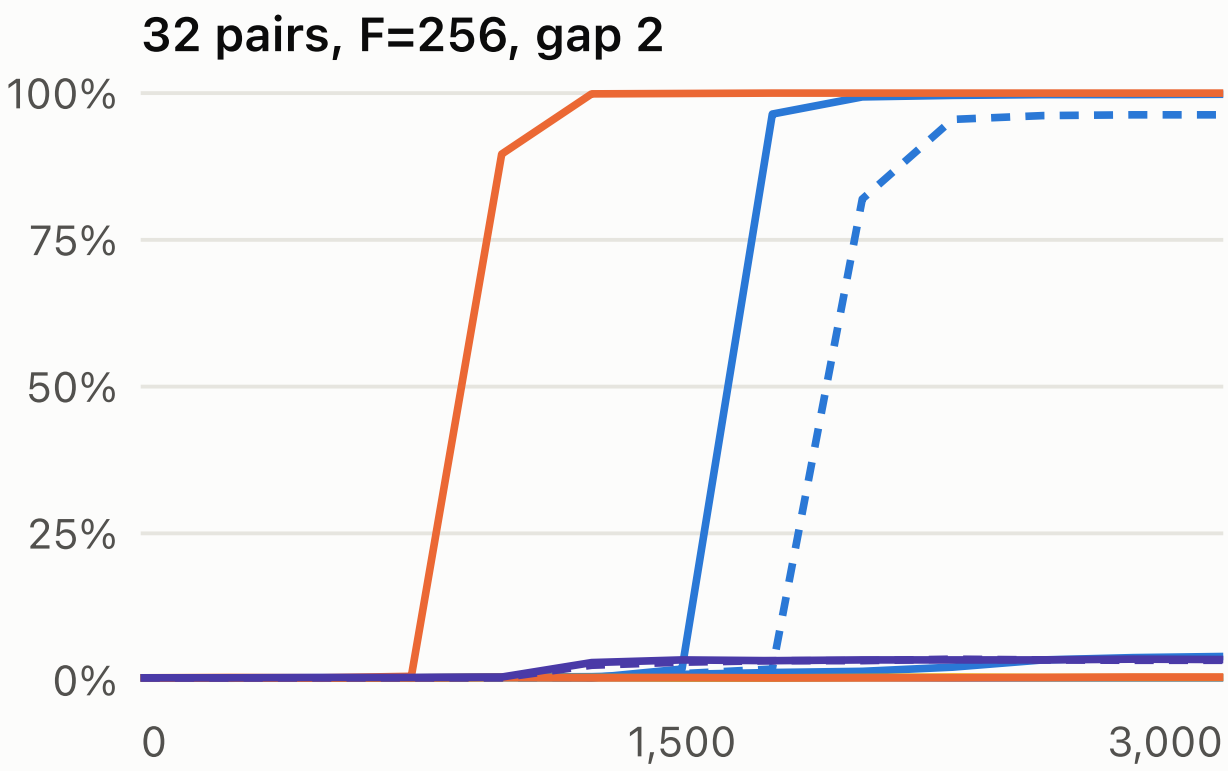
21 of 21 runs finished · 2026-10-20 02:00–06:25

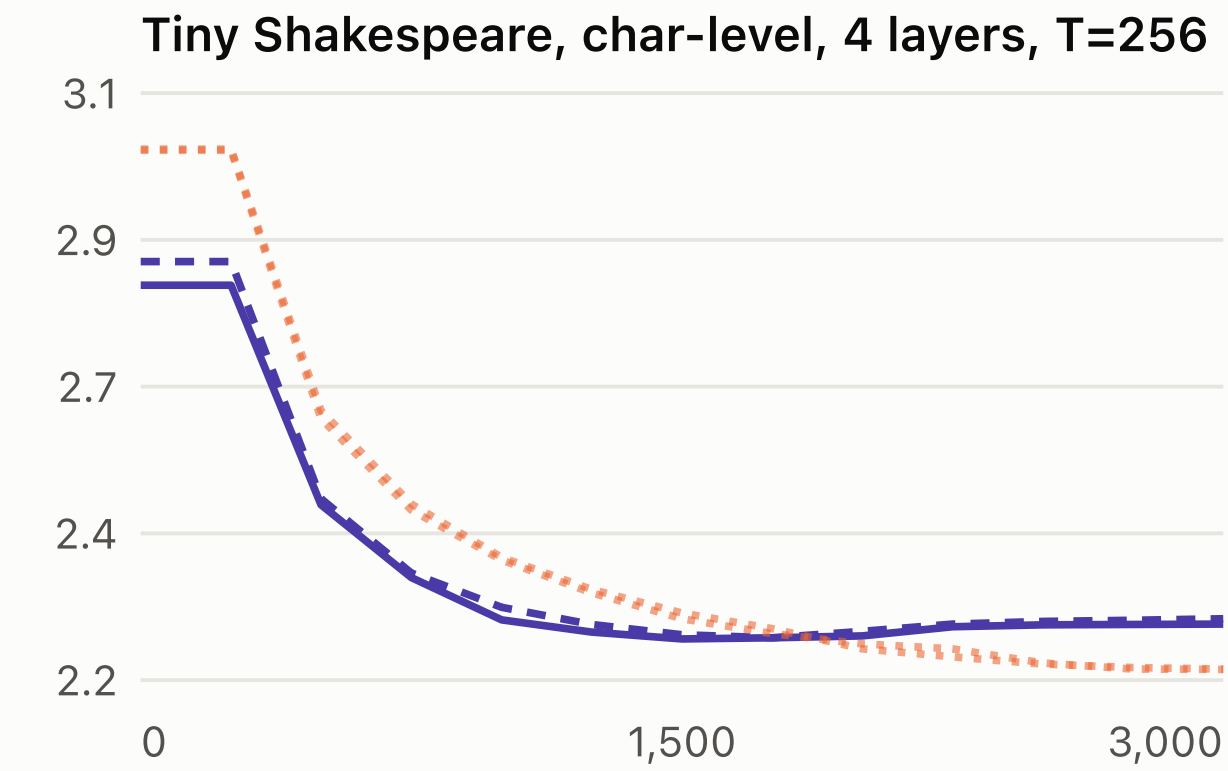
Question. Layer-wise hybrid (local window layer, then drawers) on gapped recall. Softmax with RoPE positions as the fair reference (gap task, Shakespeare). Does starting with soft addresses and hardening them over the first 30% of training fix the late click (gaps, 64 pairs)? Accuracy against memory size (64/128/256) for linear attention and drawers, paired with measured cost per token. Is the late click a learning-rate issue? One scale-up step on Shakespeare: d=256, 4 layers, 512-character context.

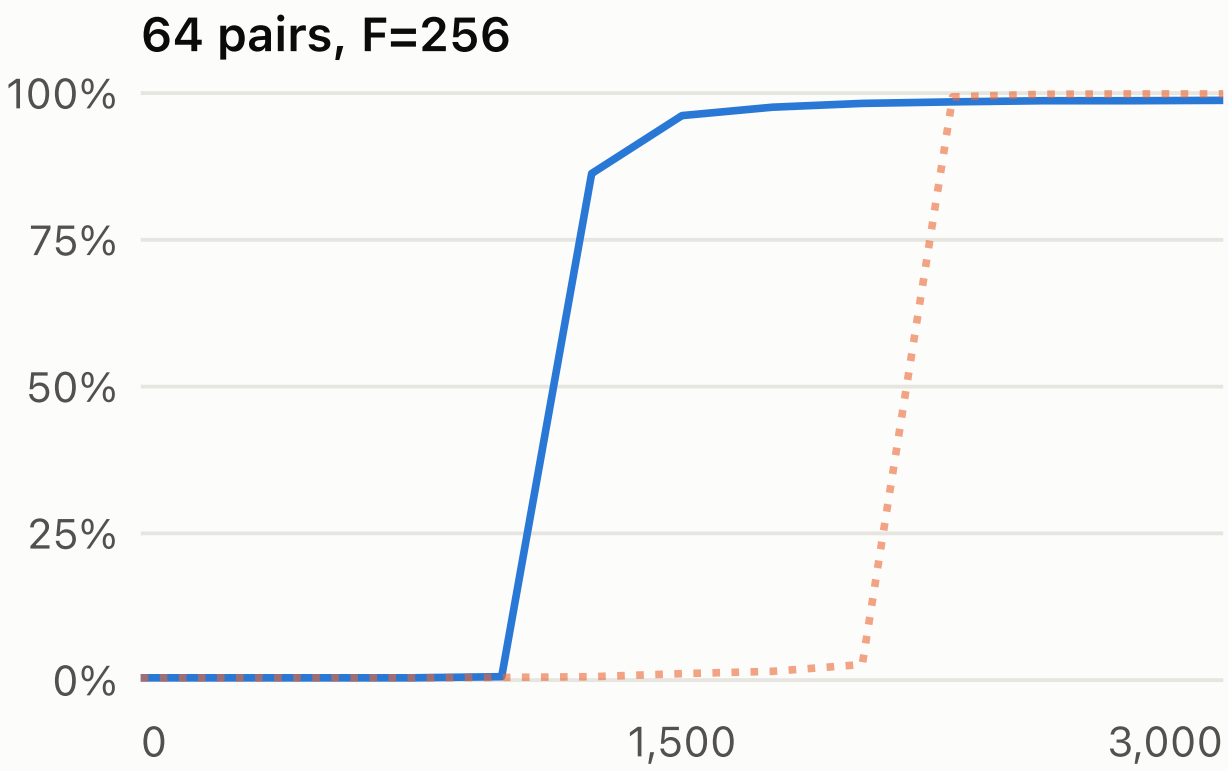
What we learned. Layer-wise hybrid (layer 1 local softmax window 16, layer 2 full-width drawers) on gapped recall: did not learn in 3,000 steps (0.46%, loss only starting to move after step 2,250); seed 1 skipped by the agreed rule. With a single drawer layer that sits on top of the local layer, the store/read agreement has to be learned on already-mixed features, and in plain drawers the matching had happened in layer 1. Both hybrid designs tried so far failed on this task; hybrids remain untested in a deeper model. Cost by memory size (CPU, batch 64, context 1,024, µs per token): linear F=32/64/128/256: 5.2 / 7.8 / 13.7 / 28.2; drawers 4×4 (F=64) 2.9, 8×4 (F=128) 4.0, 16×4 (F=256) 5.4, 4×6 (F=256) 3.0; 64×2 (F=256) 16.4. At equal memory drawers 16×4 are 2.7× (F=64) to 5.2× (F=256) cheaper; drawers 16×4 with 8× the memory cost the same as linear F=32. Fair softmax reference on

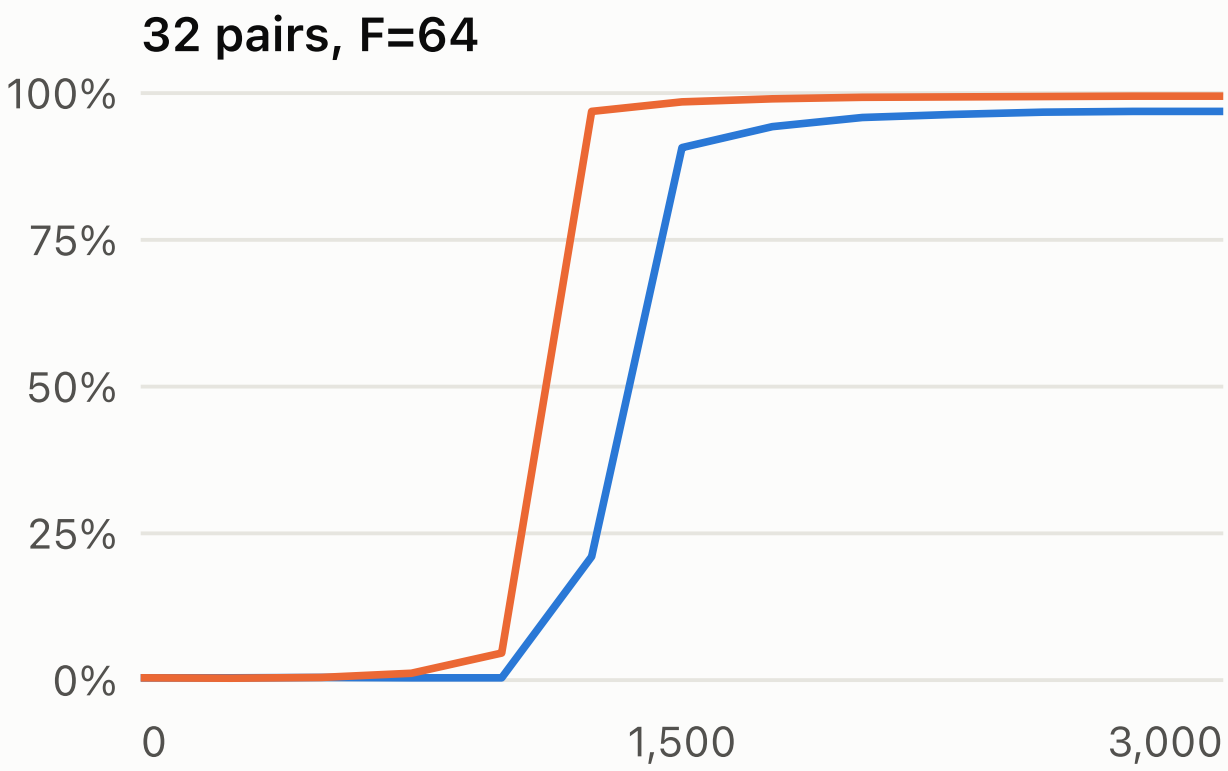
gapped recall: softmax with RoPE positions also fails (3.5% / 3.4% after 3,000 steps, two seeds), so the earlier softmax failure was not caused by missing positions. In this small 2-layer, 3,000-step setting the gapped task is solved only by linear attention (99.99%) and, partly, by drawers (90–91%); the recipe (learning rate, depth, shift width) likely disadvantages sharp-matching mixers. Softmax with RoPE on Tiny Shakespeare: 2.286 / 2.294 bpc held-out (two seeds), worse than softmax without positions (2.25), because it overfits more (training 1.56 vs 1.75). In this ~25-pass regime, held-out scores reward models that memorise less; a larger corpus is needed for a clean text comparison. **Soft-to-hard addressing on gapped recall (16 × 4): 99.79% / 96.33%** (two seeds; clicks at ~1,750 / ~2,000), against 90.0–91.2% for drawers trained hard throughout and 99.99% for linear attention. Addresses are soft for the first 900 steps and fully hard from then on; recall jumps between steps 1,500 and 1,750, with the model already in its final hard form, so deployment cost is unchanged. Soft-to-hard at 64 pairs, memory 256 (seed 0): 98.75%, clicking at ~1,250 as before; plain hard drawers reached 98.3% / 99.1%, so annealing neither helps nor hurts where hard training already learns early. Accuracy vs memory size (32 pairs, seed 0): linear 99.5% (F=64), 99.94% (F=128); drawers 4-bit 96.9% (F=64, 4 tables), 99.44% (F=128, 8 tables), 99.8% (F=256, 16 tables). Combined with the measured cost: drawers dominate the low-cost region (up to ~8 µs per token), linear attention owns the >99.9% region at 2.5–5× the cost. Learning-rate check on gapped recall (seed 0): linear attention fails at 3e-4 (0.5%), succeeds at 1e-3 (99.99%) and 3e-3 (99.997%, faster click); drawers fail at 3e-4 (0.4%) and at 3e-3 (4.0%), succeed only near 1e-3 (90–91%). Drawers have a narrower learning-rate window than linear attention; a higher rate does not fix their late click, soft-to-hard addressing does. Scale-up on Tiny Shakespeare (d=256, 4 layers, 512-character context, 4.2M parameters, softmax + RoPE): the model memorises the corpus (training 0.69 vs held-out 3.33 bpc at step 3,000; best held-out 2.30 at step 1,000), so the comparison is meaningless at this size. The linear and drawer scale-up runs were stopped; a scale-up needs a corpus much larger than 1 MB.

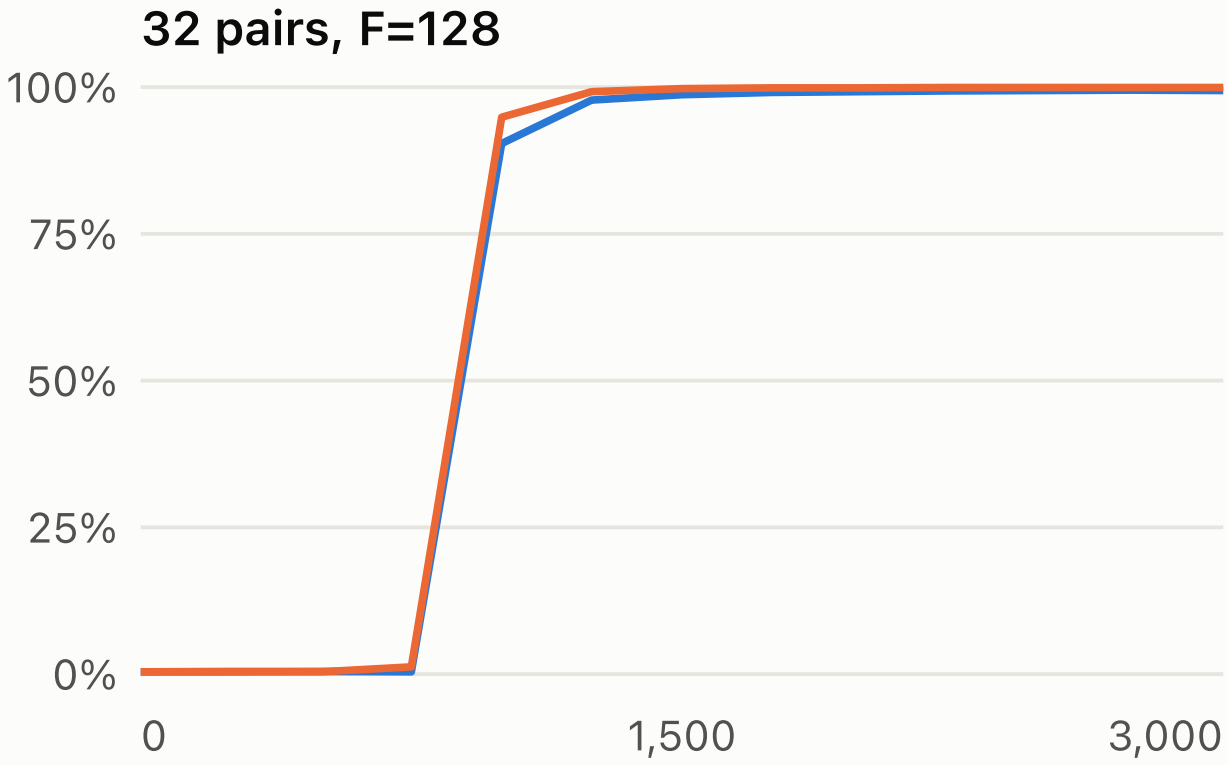


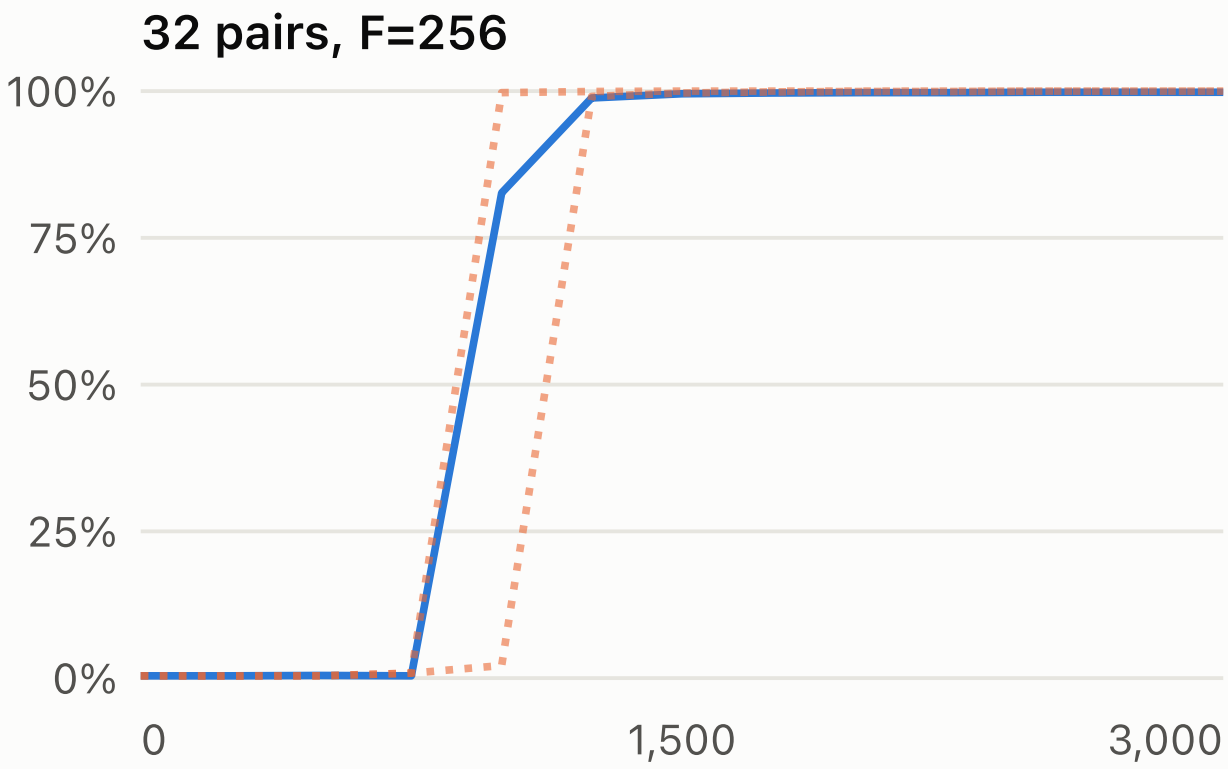

32 pairs, F=256, gap 2
100%
75%
50%
25%
0%
0
1,500
3,000

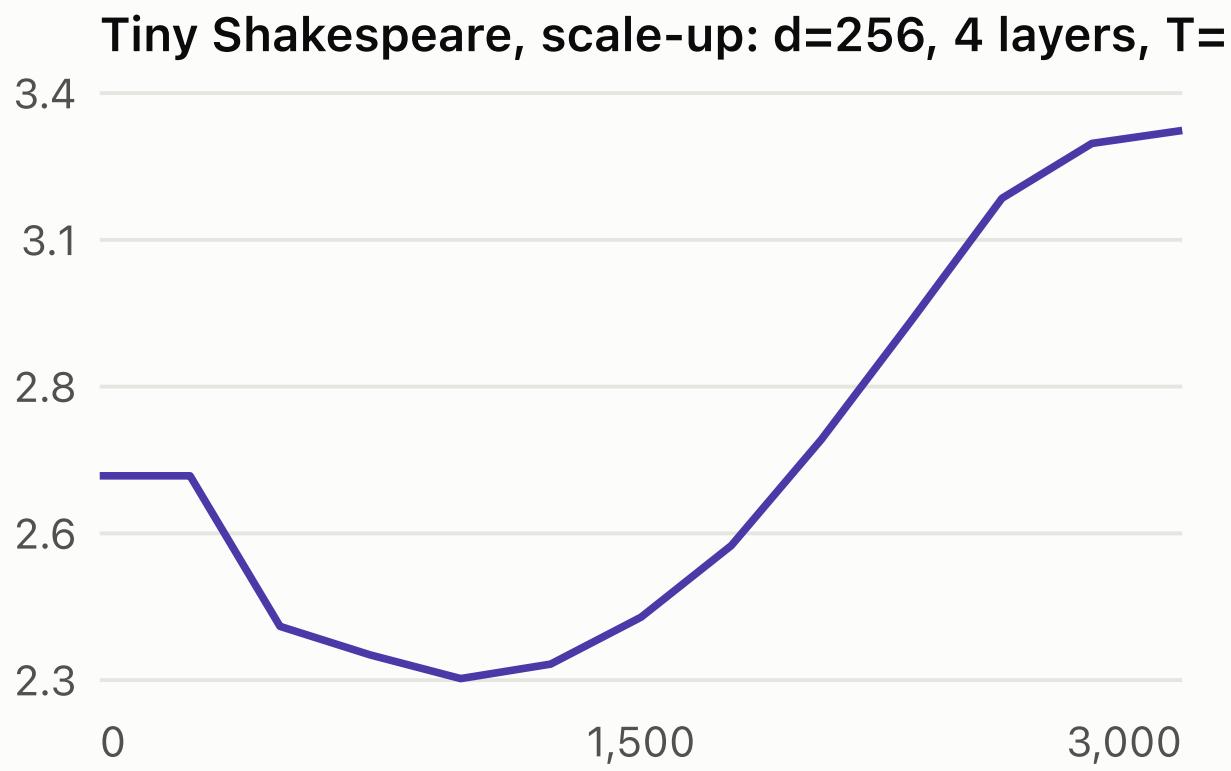

Tiny Shakespeare, char-level, 4 layers, T=256
3.1
2.9
2.7
2.4
2.2
0
1,500
3,000


64 pairs, F=256
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=64
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=128
100%
75%
50%
25%
0%
0
1,500
3,000


32 pairs, F=256
100%
75%
50%
25%
0%
0
1,500
3,000



▼ Run table

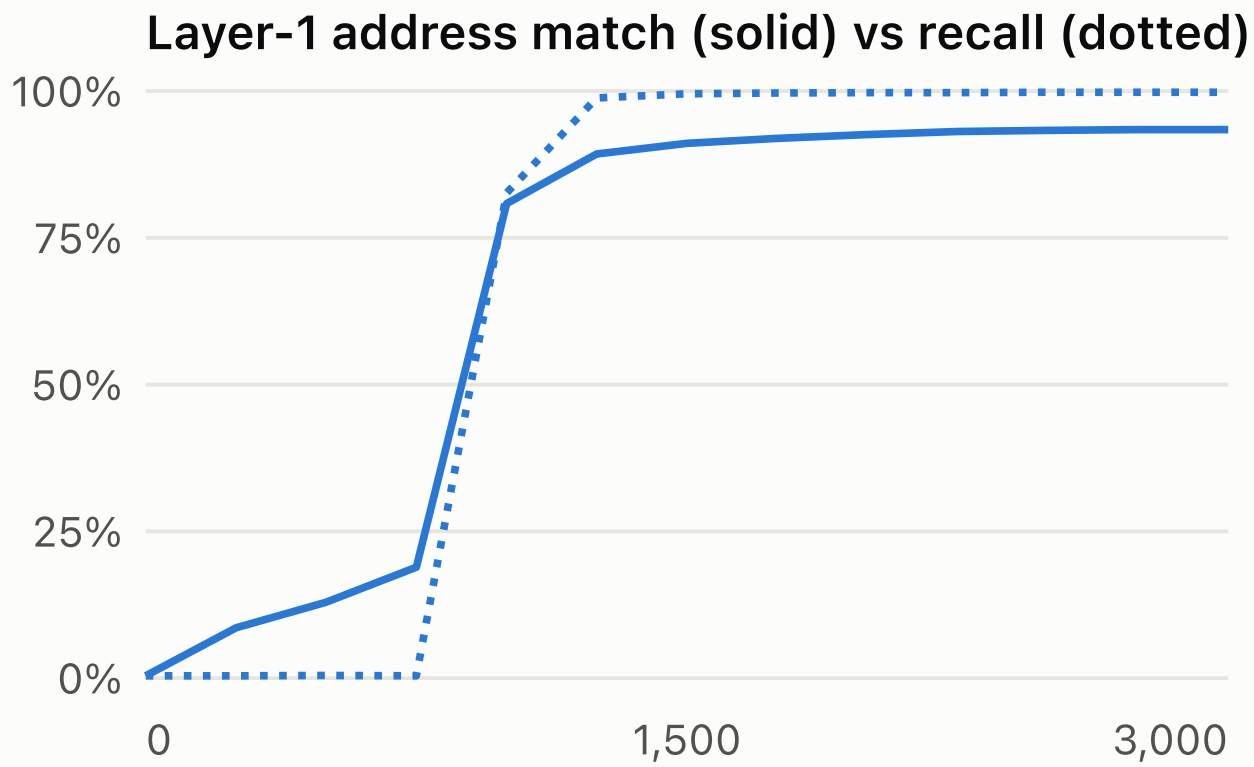
setting	model	seed	final recall	step reaching 50%	params	train min	max °C
32 pairs, F=128	drawers, hard 8×4b	0	99.44 %	1000	526,850	9.1	85.9
32 pairs, F=128	linear attention	0	99.94 %	1000	756,224	6.0	84.9
32 pairs, F=256	drawers, hard 16×4b	0	99.80 %	1000	526,850	15.0	82.7
32 pairs, F=256, gap 2	drawers, hard 16×4b, lr 0.0003	0	0.37%	never	629,250	23.6	83.7
32 pairs, F=256, gap 2	drawers, hard 16×4b, lr 0.003	0	4.04%	never	629,250	23.5	83.9
32 pairs, F=256, gap 2	drawers, hard 16×4b, soft→hard over 30% of steps	0	99.79 %	1750	629,250	24.3	82.7
32 pairs, F=256, gap 2	drawers, hard 16×4b, soft→hard over 30% of steps	1	96.33 %	2000	629,250	24.2	84.5

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
32 pairs, F=256, gap 2	layer 1 local window 16, layer 2 drawers 16×4b	0	0.46%	never	645,633	15.1	82.8
32 pairs, F=256, gap 2	layer 1 local window 16, layer 2 drawers 16×4b	1	skipped (seed 0 failed)				
32 pairs, F=256, gap 2	linear attention, lr 0.0003	0	0.52%	never	1,120,768	14.6	85.7
32 pairs, F=256, gap 2	linear attention, lr 0.003	0	100.00%	1000	1,120,768	14.9	82.5
32 pairs, F=256, gap 2	softmax attention + RoPE positions	0	3.52%	never	662,016	7.4	83.1
32 pairs, F=256, gap 2	softmax attention + RoPE positions	1	3.38%	never	662,016	7.4	81.7
32 pairs, F=64	drawers, hard 4×4b	0	96.88%	1500	526,850	6.4	82.2
32 pairs, F=64	linear attention	0	99.48%	1250	625,152	4.5	83.2
64 pairs, F=256	drawers, hard 16×4b, soft→hard over 30% of steps	0	98.75%	1250	526,850	32.0	84.9
Tiny Shakespeare, char-level, 4 layers, T=256	softmax attention + RoPE positions	0	228.60%	250	1,070,080	9.8	82.2
Tiny Shakespeare, char-level, 4 layers, T=256	softmax attention + RoPE positions	1	229.40%	250	1,070,080	9.9	84.8
Tiny Shakespeare, scale-up: d=256, 4 layers, T=512	drawers, hard 16×4b	0	skipped (seed 0 failed)				
Tiny Shakespeare, scale-up: d=256, 4 layers, T=512	linear attention	0	skipped (seed 0 failed)				

setting	model	seed	final recall	step reaching 50%	params	train min	max °C
Tiny Shakespeare, scale-up: d=256, 4 layers, T=512	softmax attention + RoPE positions	0	333.00%	250	4,237,312	24.7	83.5

Address diagnostics

Measured on 256 fresh sequences. match = query lands in the same cell as the token that stored its value (share of tables); collide = shares a cell with an unrelated pair (chance = 1/cells); read share = weight of the correct value in what the read returns.



model	layer	match	≥1 table	collide (chance)	occupancy	entropy	bit bias	bit corr	read share
drawers, hard 16×4b	1	0.935	1.000	0.062 (0.062)	0.900	0.993	0.010	0.049	0.113
drawers, hard 16×4b	2	0.400	0.994	0.107 (0.062)	0.920	0.988	0.013	0.068	0.036
drawers, hard 16×4b	1	0.935	1.000	0.062 (0.062)	0.900	0.993	0.010	0.049	0.113

model	layer	match	≥1 table	collide (chance)	occupancy	entropy	bit bias	bit corr	read share
drawers, hard 16×4b	2	0.400	0.994	0.107 (0.062)	0.920	0.988	0.013	0.068	0.036

▶ Generated text samples (seed 0, temperature 0.8)

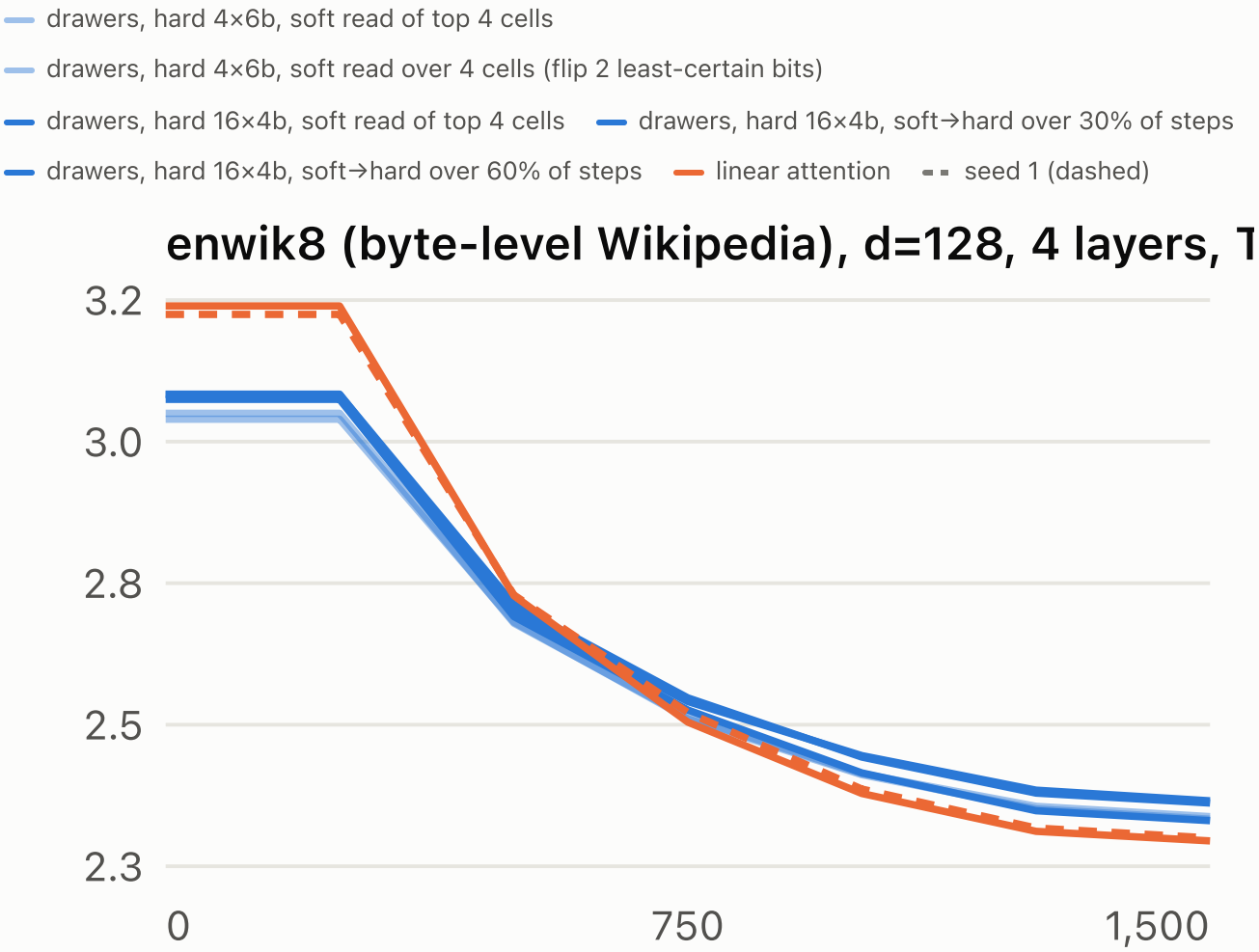
Batch 8 — enwik8 pilot: is a scale-up worth 8 hours? + bounded soft reads

7 of 11 runs finished · 2026-10-20 07:45–10:00

Question. Small model (d=128, 4 layers, 256 bytes context), 1,500 steps on enwik8 (90 MB of Wikipedia, each byte seen ~0.06 times): linear attention vs drawers 16×4 with soft-to-hard addressing. Pre-registered: go for the 8-hour scale-up if drawers are within 0.05 bpc of linear, both still improving, and the train/held-out gap is below 0.05 bpc; no-go if drawers trail by more than 0.10. Follow-up idea (bounded soft read): per table, read and write the 4 most likely cells weighted by their probability instead of one hard cell; cost grows with 4, not with table size. Tested as 16 × 4 bits (4 of 16 cells, 64 cells touched per head) and 4 × 6 bits (4 of 64 cells, 16 cells touched, like 16 × 4 hard), plus a cheaper way to pick the 4 cells (flip the 2 least-certain bits).

What we learned. enwik8, 1,500 steps, seed 0: linear attention 2.341 bpc held-out (training 2.321), drawers 16×4 with soft-to-hard (30%) 2.400 (training 2.380). No overfitting (train/held-out gap 0.02). Drawers are ahead while addresses are soft (3.050 vs 3.191 at step 250) and fall behind once they are fully hard (from step ~450), ending 0.060 bpc behind. By the pre-registered rule this is the in-between zone (0.05–0.10): one more cheap check before any 8-hour run. Check chosen: a longer soft phase (60% of training). Result of the check: soft for 60% of training gives 2.404 bpc, no better than 30% (2.400); the gap to linear attention (0.06) opens around step 750 even while addresses are still mostly soft, so it is not just the hard switch. Decision by the pre-registered rule: no 8-hour scale-up on the M1. At this size drawers trail linear attention on varied real text by ~0.06 bpc (2.5%) at ~5× lower measured decode cost. Bounded soft read (top-4 cells per table, 16 × 4 bits, seed 0): 2.372 bpc held-out, cutting the gap to linear attention from 0.060 to 0.032 (inside the pre-registered 'go' threshold of 0.05); best of all models up to step 500. Equal-cost variant (top-4 of 64 cells, 4 tables × 6 bits: touches 16 cells per head, like 16 × 4 hard): 2.375 bpc, gap 0.034, the same as the 2×-cost 16 × 4 top-4 (2.372); level with linear attention at step 750 (2.534 vs 2.529), the gap reopens to ~0.03 in the second half. Cheap bounded soft read (4 × 6 bits; per table the cell plus the 3 cells from flipping the 2

least-certain bits, weighted exp(−flipped margin/τ)): 2.379 bpc, gap 0.038, within 0.002–0.004 of the exact top-4 at every checkpoint. Measured cost 10.9 µs per token vs 13.2 for exact top-4, 5.5 for hard drawers and 29.5 for linear attention: the saving from not scoring all cells is small; most of the extra cost is the weighted scatter/gather of 4 rows per table and extra small operations, which a fused kernel would largely remove (not tested).



▼ Run table

setting	model	seed	final val bpc	best	params	train min	max °C
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	drawers, hard 16×4b, soft read of top 4 cells	0	2.372	2.372	1,053,444	24.0	89.4
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	drawers, hard 16×4b, soft→hard over 30% of steps	0	2.400	2.400	1,053,444	14.4	85.2
enwik8 (byte-level Wikipedia), d=128, 4	drawers, hard 16×4b, soft→hard over 60% of	0	2.404	2.404	1,053,444	14.7	86.0

setting	model	seed	final val bpc	best	params	train min	max °C
layers, T=256	steps						
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	drawers, hard 4×6b, soft read of top 4 cells	0	2.375	2.375	1,053,444	11.8	93.8
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	drawers, hard 4×6b, soft read over 4 cells (flip 2 least-certain bits)	0	2.379	2.379	1,053,444	11.4	94.3
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	linear attention	0	2.341	2.341	2,036,480	8.2	90.1
enwik8 (byte-level Wikipedia), d=128, 4 layers, T=256	linear attention	1	2.345	2.345	2,036,480	8.2	88.6

▶ Generated text samples (seed 0, temperature 0.8)

Glossary

Drawer memory — this project's name for a hash-addressed associative memory written at inference; in the plain-language analogy, tables are cabinets and cells are drawers.

MQAR — multi-query associative recall: a sequence of random key→value pairs, then the keys again; the model must name each key's value. **Recall** — share of those answers that are correct (chance 0.4%). **Memory F** — memory size per head: cells (drawers) for drawer memory, feature dimension for linear attention; equal F means equal recurrent state. **Tables × bits** — number of tables (cabinets) and yes/no questions per table; cells per table = 2^bits. **Neighbour cell** — also read the cell one answer away from the least certain question. **Confidence** — down-weight tables whose answers were close calls. **Step reaching 50%** — training step at which recall first exceeded 50%, our measure of when learning 'clicks'. **bpc** — bits per character on held-out text, lower is better.

Soft-to-hard addressing — training trick: cells are chosen softly (a weighted mix over all cells of a table) at the start and harden gradually; after 30% of training the model is fully hard, the cheap form used at inference. **Bounded soft read** — inference-time variant: each token writes to and reads from its 4 most likely cells per table, weighted by their probability (exact top-4, or the cheaper 'flip-2': the cell plus those reached by flipping the 2 least-certain bits). **Hybrid** —

a model mixing a local softmax window (last 16 tokens) with drawer memory. **enwik8** — the first 100 MB of English Wikipedia, a standard byte-level language-modelling benchmark; 90 MB for training, 5 MB held out. **µs per token** — measured time for one attention-replacement layer to process one new token during generation (CPU, PyTorch, 64 sequences at a time).